ASSET MANAGEMENT SYSTEM

MAIN PROJECT REPORT

Submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

2023 - 26

Done By,

Rasha Febin (Reg. No: 230021080553)

Under the guidance of

Mr. Joby Jacob

RAJAGIRI COLLEGE OF MANAGEMENT AND APPLIED SCIENCES

(Affiliated to Mahatma Gandhi University, Kottayam)

Rajagiri Valley P.O, KERALA- 682039

RAJAGIRI COLLEGE OF MANAGEMENT AND APPLIED SCIENCES

(Affiliated to Mahatma Gandhi University, Kottayam)

Rajagiri Valley P.O, KERALA- 682039

CERTIFICATE

*This is to certify that the mini project work titled "**ASSET MANAGEMENT SYSTEM** "submitted to Mahatma Gandhi University in partial fulfillment of the requirements for the award of the Degree of Bachelor Of Computer Applications is a record of the original work done by **Rasha Febin** under my supervision and guidance and that this project work has not formed the basis for the award of any Degree or similar title to any candidate of this or any other University.*

Faculty In-Charge **Project Coordinator** **Head of the Department**

Submitted for viva-voce held on ……../……../……

Internal Examiner **External Examiner**

RAJAGIRI COLLEGE OF MANAGEMENT AND APPLIED SCIENCES

(Affiliated to Mahatma Gandhi University, Kottayam)

Rajagiri Valley P.O, KERALA- 682039

DECLARATION

I **RASHA FEBIN** hereby declare that project report entitled "**ASSET MANAGEMENT SYSTEM"** submitted in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Applications is a record of original research work done by me under the supervision & guidance of **Mr. JOBY JACOB** and the dissertation has not formed the basis for the award of any Degree or similar title to any candidate of this or any other University.

Rasha Febin

Place: Kakkanad

Date:…./……/……

ACKNOWLEDGEMENT

I consider it as a privilege to express my sincere gratitude and respect to all those who guided and inspired me in the successful completion of this project work.

I convey my reverential salutation to **Almighty God**, for enabling me to take up and complete the project successfully.

I would like to express my sincere thanks to **Rev. Fr. Ajeesh Puthussery CMI,** Director and **Prof. Dr. Laly Mathew,** Principal, Rajagiri College of Management and Applied Sciences for providing the necessary infrastructure and support for the completion of this project work.

I would like to express my sincere thanks to **Ms. Nileena Sasidharan**, HOD, PG Department of Computer Science, Rajagiri College of Management and Applied Sciences for her valuable advice and support which have helped me greatly in the accomplishment of the project.

I would like to express my sincere thanks to the Guide and Project Coordinator **Mr. Joby Jacob**, Assistant Professor, PG Department of Computer Science, Rajagiri College of Management and Applied Sciences for his consistent guidance and inspiration throughout the period for the completion of my project.

I would like to thank all the teaching and non-teaching staff of Rajagiri College of Management and Applied Sciences, for their valuable guidance and suggestions rendered during the project.

Finally, I thank my parents and all my friends for their help, encouragement and moral support given to me during the course of this work.

Rasha Febin

TABLE OF CONTENTS

1. INTRODUCTION ……..……..…..…….. 1

1.1. About the Project…….…….……..…….……. 1

2. REQUIREMENT ANALYSIS AND SPECIFICATION……..……... 2

2.1. System Study..…….… 2

2.1.1. Existing system ……..…….…….…...……. 2

2.1.2. Proposed system….……..….…… 3

2.1.3. Feasibility study……...……. 4

2.1.3.1.Technical feasibility…….……...…..……. 5

2.1.3.2.Economical feasibility……...……..…...…... 5

2.1.3.3.Operational feasibility……...…….…..... 6

2.2. User characteristics…….....….... 7

2.3. System specification……...……...…….…. 9

2.3.1. Hardware specification ……..…….. 9

2.3.2. Software specification …….……..…….. 9

2.3.3. About the software tools and platforms…….…... 10

3. SYSTEM MODELING……...…….. 11

3.1. Modules and description……...…….. 11

3.2. Data Flow Diagram ……...……. 15

3.3. Entity relationship diagram. ……... 23

4. SYSTEM DESIGN……...……...….. 25

4.1. Input design……..…….……. 25

4.2. Output design…….. 30

4.3. Database design…….. 36

5. TESTING……..……..…….…. 41

5.1. Introduction……...……. 41

5.2. Test cases……...……...…..…. 43

6. IMPLEMENTATION……..……...…….....… 46

6.1. Introduction……..……....….…. 46

6.2. Installation procedure…..……..…...……. 47

6.3. Implementation plan…….…..……....… 48

7. CONCLUSION…….……..…….……...….. 49

7.1. Future Enhancement…….…..……....… 49

BIBLIOGRAPHY……....…….…….. 50

APPENDICES

APPENDIX A……....…. 52

Sample source code/pseudo code

APPENDIX B……...…….…...…. 76

Acronyms

1.1 ABOUT THE PROJECT

The Asset Management System is a web-based application designed to simplify and automate the management of assets in organizations such as colleges, hospitals, and offices. The project provides a centralized platform for registering, tracking, and monitoring a wide range of assets—including equipment, furniture, computers, and machinery—across different locations and categories.

The system utilizes QR code technology to uniquely identify every asset. When an asset is added, a QR code is automatically generated and linked to its details. Users can scan the QR code with a mobile device or scanner to instantly view asset information—such as location, purchase and expiry dates, warranty, responsible staff, and current status (e.g., active or scrapped).

Key modules include user management, category management, asset management, QR code generation, scan and view functionality, notifications and feedback for maintenance or issues, and comprehensive report generation. Assets can be added individually or imported in bulk via Excel upload, enabling faster onboarding and record-keeping.

Designed with both administrative and general users in mind, the system ensures secure access based on user roles, supports feedback management, and sends notifications for asset expiry or required actions. By automating asset tracking and reporting, the platform eliminates manual errors, improves accountability, and enhances operational efficiency for any organization.

1.2 ABOUT THE ORGANIZATION

The Asset Management System is a centralized, web-based platform developed to support efficient tracking and management of institutional assets at Rajagiri College of Management and Applied Sciences. The system is designed to manage a wide range of assets including computers, printers, furniture, and other equipment used within the institution. It functions as a digital repository that captures, stores, and manages real-time asset information, ensuring accuracy, reliability, and ease of access.

One of the primary objectives of the system is to streamline asset tracking by maintaining detailed records for each item, including product specifications, purchase details, cost, warranty or expiry dates, assigned custodian, and current location. By digitizing these records, the system eliminates the need for manual documentation, reduces the risk of data loss, and enhances overall operational efficiency within the college.

A key feature of the Asset Management System is the integration of QR code technology, where each asset is assigned a unique QR code. This allows users to quickly retrieve asset details through scanning, making identification and verification faster and more convenient. The system also implements role-based access control, enabling administrators to manage and monitor all assets across the institution, while regular users can access and view only the assets assigned to them.

The platform further supports effective asset lifecycle management by providing insights into ageing assets, upcoming expirations, and items that may require maintenance, replacement, or disposal. This helps the institution make informed decisions regarding resource utilization and budgeting.

Additionally, the system enhances usability by offering features such as exporting asset data into spreadsheet formats for reporting and analysis, as well as feedback and comment mechanisms that allow users to report issues or share observations. A free QR code generation tool is also included, making the system more flexible and user-friendly

2.1 SYSTEM STUDY

The traditional method of asset management in many organizations is mostly manual and unstructured. Asset details are commonly maintained using registers, spreadsheets, or standalone files, which makes the process time-consuming and error-prone. Tracking assets, monitoring their usage, checking expiry or warranty details, and maintaining accountability becomes difficult when records are scattered or outdated. There is no quick way to identify assets or verify their details, especially during audits or physical verification.

In the existing approach, administrators face challenges in managing large volumes of assets efficiently. Manual record-keeping often leads to duplication of data, loss of information, and difficulty in generating accurate reports. Users also lack a transparent system to view asset details or updates. The absence of automation makes asset tracking slow and unreliable.

The proposed Asset Management System is designed to overcome these limitations by providing a centralized, web-based platform for managing assets efficiently. The system allows secure user authentication, role-based access, asset categorization, QR code generation, and real-time tracking of asset information. By digitizing asset records and automating key processes, the system improves accuracy, transparency, and ease of management.

A feasibility study was conducted to ensure that the proposed system is practical from technical, economic, and operational perspectives. The system uses open-source technologies such as Python Flask and MySQL, making it cost-effective and easy to maintain. The interface is user-friendly and can be operated by users with basic computer knowledge.

2.1.1 Existing System

In the existing system, asset management is carried out using manual methods or basic digital tools like spreadsheets. Asset details such as product name, category, purchase date, expiry date, and location are recorded

manually, which increases the risk of errors and data inconsistency. Tracking assets over time becomes difficult, especially when the number of assets increases.

There is no efficient mechanism to uniquely identify each asset. Physical verification requires checking registers or files, which is time-consuming. Updating asset information or generating reports requires manual effort, and there is no real-time visibility into asset status or expiry. Administrators find it difficult to monitor assets effectively, and users have limited access to asset-related information. Overall, the existing system lacks automation, transparency, and reliability.

Disadvantages of existing system

- Manual and time-consuming asset record maintenance
- High chance of data duplication and errors
- No quick method to identify or verify assets
- Difficult to track asset expiry and warranty details
- Lack of centralized data and reporting
- Limited transparency and accountability

2.1.2 Proposed System

The proposed system is a web-based Asset Management System developed to automate and streamline the process of managing organizational assets. The system provides secure login functionality and role-based access for administrators and users. Administrators can add and manage asset categories, register assets, generate QR codes for each asset, and monitor asset details efficiently.

Each asset is assigned a unique QR code that stores essential asset information. This enables quick identification and verification of assets by scanning the QR code, reducing manual effort during audits or inspections. The system also

tracks important details such as purchase date, expiry date, warranty period, location, and custodian information.

Users can view asset details through the dashboard, while administrators have full control over asset records, reports, and system data. The system stores all information in a centralized database, ensuring data consistency and easy retrieval. By automating asset tracking and reporting, the proposed system improves efficiency, accuracy, and transparency in asset management.

Advantages of proposed system

- Centralized and organized asset data management
- QR code-based asset identification and tracking
- Reduced manual work and human errors
- Easy monitoring of asset expiry and warranty
- Secure, role-based access control
- Quick report generation and improved transparency

2.1.3. Feasibility Study

Before the development of the Asset Management System, a feasibility study was conducted to analyze whether the proposed system is practical, cost-effective, and suitable for real-world implementation. This study helps in understanding the technical requirements, economic factors, and operational aspects involved in developing and maintaining the system. The feasibility study ensures that the system can be successfully implemented and efficiently used by the intended users.

The feasibility study mainly focuses on determining whether the proposed system is feasible or not. There are three major aspects considered in this study:

- Technical Feasibility
- Economic feasibility

- Operational feasibility

2.1.3.1. Technical Feasibility

The proposed Asset Management System is technically feasible as it is developed using well-established and widely supported technologies such as Python Flask, MySQL, HTML, CSS, and JavaScript. These technologies are open-source, reliable, and suitable for developing web-based applications. The system does not require any specialized hardware or complex technical infrastructure.

The application can be executed on a local server or deployed on a cloud-based environment. The use of Flask framework ensures smooth handling of server-side logic, routing, and database interaction. QR code generation is implemented using standard Python libraries, making the feature easy to integrate and maintain. Since the development tools and technologies are already available and familiar, the system is technically feasible to implement and operate.

2.1.3.2. Economic Feasibility

Economic feasibility evaluates the cost involved in developing and maintaining the system. The Asset Management System is economically feasible as it uses open-source software tools such as Python, Flask, and MySQL, which do not require any licensing fees. The system can be developed using a standard personal computer, minimizing hardware costs.

By automating asset tracking, record maintenance, and report generation, the system reduces manual work and paperwork. This leads to long-term cost savings by minimizing human effort and reducing errors. Since maintenance costs are low and no expensive infrastructure is required, the proposed system is cost-effective and economically feasible.

2.1.3.3. Operational Feasibility

Economic feasibility evaluates the cost involved in developing and maintaining the system. The Asset Management System is economically feasible as it uses open-source software tools such as Python, Flask, and MySQL, which do not require any licensing fees. The system can be developed using a standard personal computer, minimizing hardware costs.

By automating asset tracking, record maintenance, and report generation, the system reduces manual work and paperwork. This leads to long-term cost savings by minimizing human effort and reducing errors. Since maintenance costs are low and no expensive infrastructure is required, the proposed system is cost-effective and economically feasible.

2.2 USER CHARACTERISTICS

The Asset Management System is designed to support different types of users, each having specific roles, responsibilities, and access permissions. Understanding the characteristics of users is important to ensure that the system is easy to use, efficient, and capable of meeting the needs of all stakeholders. The users of the system differ in their level of technical knowledge, frequency of system usage, and purpose of interaction.

The system follows a role-based access approach, where each user is provided with features relevant only to their role. This helps in maintaining security, simplicity, and clarity while using the system. The interface is designed to be user-friendly so that users with basic computer knowledge can operate the system without difficulty.

The Asset Management System mainly consists of the following user categories:

2.2.1 Administrator

2.2.2 Users

2.2.1 Administrator

Administrators are responsible for managing and overseeing the entire asset management system. They possess the highest level of access and control, which allows them to:

- Register and approve users, including external organizations.
- Create, edit, or delete asset categories.
- Add, update, and remove asset records, individually or via Excel import.
- Generate and assign QR codes to assets.

- View and respond to notifications and feedback submitted by users.
- Generate summaries and detailed reports on assets, usage, and status. Administrators are expected to be comfortable with web-based dashboards and capable of handling core backend operations and system configurations.

2.2.2 Users

Users are individuals who access the system primarily to view and interact with asset-related information. After successful registration and login, users can access the user dashboard, where they can view asset details such as asset name, category, location, status, and QR code information.

Users can scan QR codes to quickly retrieve asset information, which helps in easy identification and verification of assets. The user interface is kept simple and intuitive to ensure that users with minimal technical skills can easily navigate the system. Users do not have administrative privileges and can only access features permitted to their role, ensuring system security and controlled access.

2.3 SYSTEM SPECIFICATION

2.3.1 Hardware Specification

The hardware specification outlines the physical resources required to run the Asset Management System. The system does not require any specialized hardware and can operate smoothly on basic computer systems.
The hardware required for the system is listed below:

Processor	Intel Core i5 or above
RAM	4 GB or above
Storage	Minimum 250 GB of storage
Other	Keyboard and Mouse

2.3.2 Software Specification

Operating System	Windows 10 or Linux-based OS
Front End	PHP, HTML, CSS, JavaScript
Back End	MySQL
Web Server	Flask Development Server / Apache

2.3.3 About software tools and platform

FRONT-END SPECIFICATION:

The frontend of the Asset Management System provides the interface through which users interact with the system. It is developed using HTML, CSS, and JavaScript to ensure a responsive and user-friendly design.

HTML is used to structure the content of the web pages such as forms, tables, dashboards, and navigation menus. CSS is used to enhance the visual

appearance of the application, providing a clean and consistent layout across all pages. JavaScript is used to add interactivity and client-side validation, ensuring that user inputs are accurate before submission.

The frontend dynamically displays content based on user roles, ensuring that administrators and users see only the features relevant to them.

BACK-END SPECIFICATION: MYSQL

The backend of the Asset Management System is developed using Python with the Flask framework. Flask handles routing, server-side logic, session management, and communication with the database. The backend is responsible for processing user requests, managing asset data, generating QR codes, and enforcing role-based access control.

The database used in the system is MySQL, which stores all information related to users, assets, categories, QR codes, feedback, and reports. MySQL ensures data consistency, integrity, and efficient data retrieval. The use of a relational database allows proper relationships between tables using primary and foreign keys, making data management reliable and structured.

3.1 MODULES AND DESCRIPTION

The Asset Management System is divided into several functional modules, each designed to handle a specific task within the system. These modules work together to ensure smooth operation, better organization, and efficient management of assets. By dividing the system into modules, the application becomes easier to understand, maintain, and enhance.

Each module is designed based on the roles of users and the operational needs of asset tracking. The major modules included in the system are listed below:

1. User Management
2. Category Management
3. Asset Management
4. QR Generation
5. Scan and View Assets
6. Feedback
7. Report Generation
8. Asset Expiry / Disposal

1. USER MANAGEMENT

This module handles user registration, authentication, and access control within the system. It ensures that only authorized users can access the application based on their assigned roles. Users can register and log in securely using their credentials, while administrators have the authority to manage user accounts.

The administrator can assign roles and control access permissions. Role-based access ensures that only authorized users can perform sensitive operations such as managing assets and categories. This module plays a key role in maintaining system security and proper authorization.

2. CATEGORY MANAGEMENT

The Category Management module allows administrators to create, update, and organize asset categories such as IT Equipment, Furniture, and Lab Equipment. Categories help in classifying assets in a structured manner, making asset management more organized and efficient.
By grouping assets under appropriate categories, the system enables easier searching, filtering, and reporting. This module helps improve clarity and simplifies asset tracking across different departments or locations.

3. ASSET MANAGEMENT

The Asset Management module is the core component of the system. It centralizes all asset-related records and allows users to add, update, view, and manage assets efficiently. Assets can be added either individually through forms or in bulk using file uploads.

Each asset is assigned a unique identification code and linked with important details such as category, location, purchase date, expiry date, warranty period, custodian, and department. This module ensures accurate record-keeping and helps maintain a complete lifecycle history of assets.

4. QR GENERATION

This module is responsible for generating a unique QR code for every asset in the system. The QR code contains encoded information such as the asset ID or a secure reference link associated with the asset.

QR generation enables quick identification of assets and eliminates the need for manual searching. The generated QR codes can be printed and attached to physical assets, making asset verification faster and more reliable.

5. SCAN AND VIEW ASSETS

The Scan and View Assets module allows users to scan QR codes using a scanner or camera-enabled device. Upon scanning, the system retrieves and displays the corresponding asset details instantly.

This feature ensures quick verification of asset information and significantly reduces manual errors. It is especially useful during audits, inspections, and physical asset verification processes.

6. FEEDBACK

The Feedback module allows users to submit feedback or issues related to assets or system usage. Users can report problems such as incorrect asset details, damaged assets, or system-related concerns.

The submitted feedback is stored in the database and can be viewed by administrators through the admin dashboard. This module improves communication between users and administrators and helps in maintaining system accuracy and reliability.

7. REPORT GENERATION

The Report Generation module helps administrators generate detailed and summary reports related to assets, categories, asset status, and usage history. Reports provide valuable insights into asset distribution, availability, and condition.

These reports assist administrators in decision-making, inventory planning, auditing, and ensuring accountability. Reports can be generated in digital formats for easy storage and reference.

8. **ASSET EXPIRY\DISPOSAL**

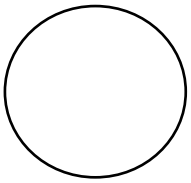
This module manages assets that have reached the end of their lifecycle. It tracks asset expiry dates and identifies assets that are no longer usable or have exceeded their warranty or operational period.

The system records disposal details such as disposal method (sold, donated, scrapped, or stored), approval information, and responsible authority. This module ensures proper documentation and accountability in asset disposal and helps maintain accurate asset records.

3.2 DATA FLOW DIAGRAM

A data flow diagram (DFD) is a graphical tool used to illustrate and analyze the movement of data within a system. It serves as a fundamental tool from which other components of the system are developed. DFDs describe the logical flow of data between various elements in a system, such as people, departments, and workstations, independently of the physical components. These diagrams, often referred to as "bubble charts," are crucial for understanding system requirements and identifying key data transformations that will be translated into programs during system design. DFDs consist of interconnected bubbles representing processes, data sources or destinations, data stores, and information or data lines.

The four symbols used for drawing DFDs are:



Process or (Function)

External Entity

Data Store

Data Flow

Rules for drawing data flow diagrams:

Rule 1: Define the context of the diagram by identifying all input and output data flows.

Rule 2: Choose a starting point for drawing the DFD.

Rule 3: Provide meaningful labels for all data flow lines.

Rule 4: Label processes with action verbs that relate to input and output data flows.

Rule 5: Exclude insignificant functions usually handled during programming.

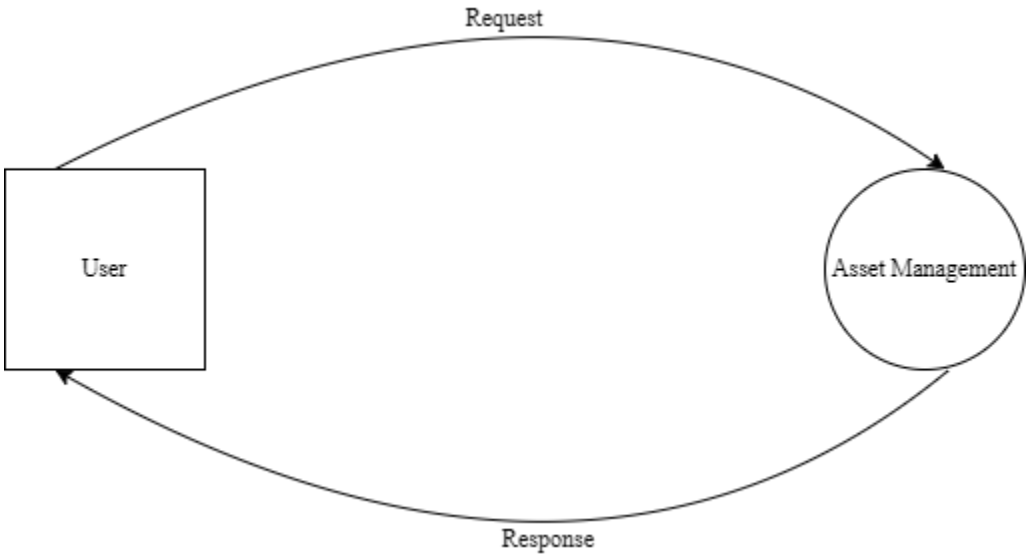
Rule 6: Avoid including control or flow of control information.

Rule 7: Avoid overcrowding the DFD with excessive information.

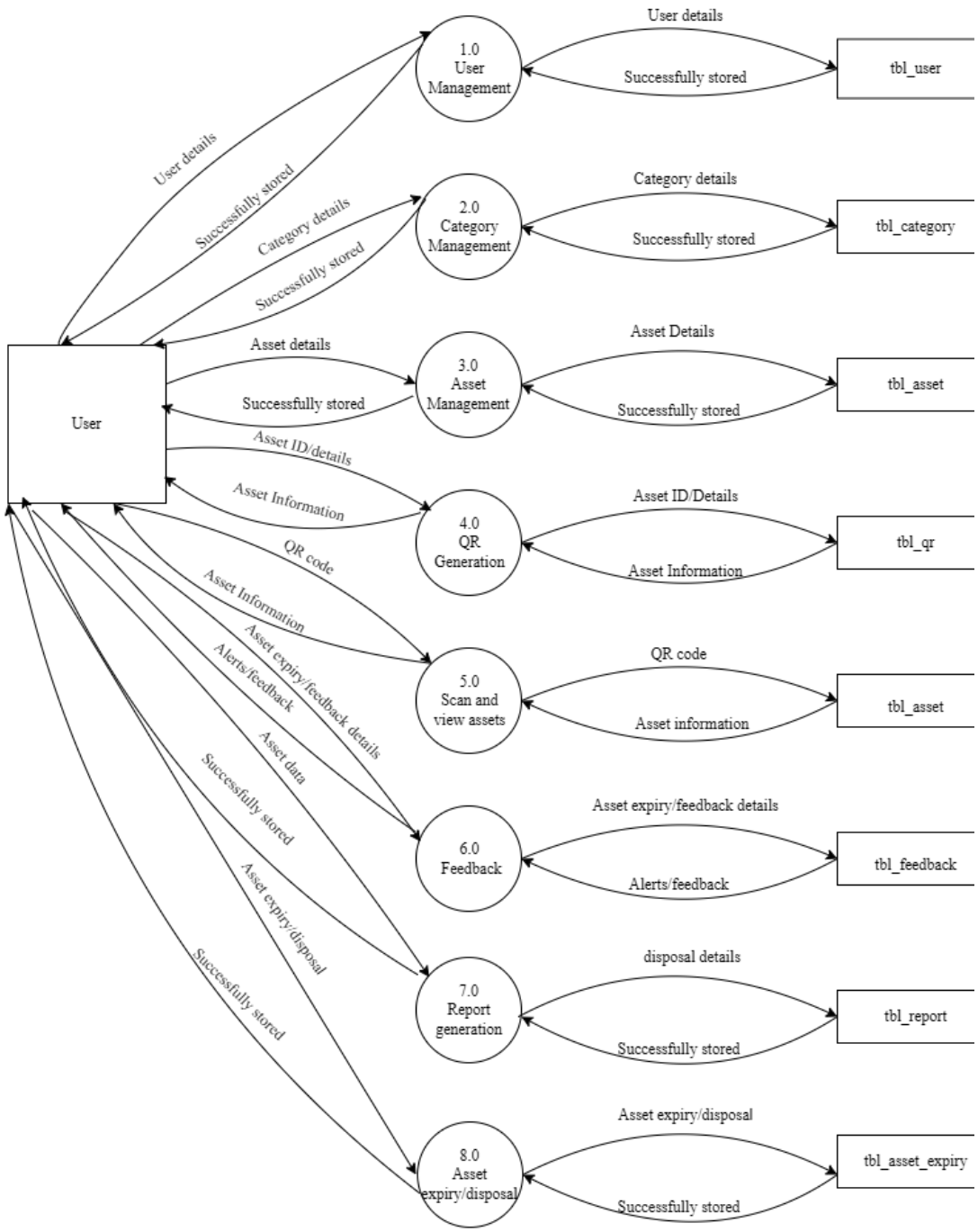
Rule 8: Be open to starting the diagram over if necessary.

DATA FLOW DIAGRAMS

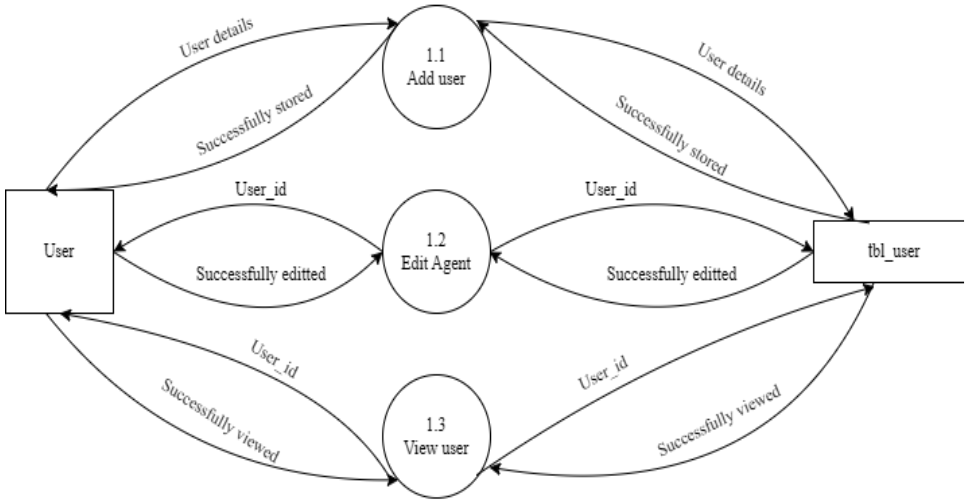
Level 0 Showing Asset Management System



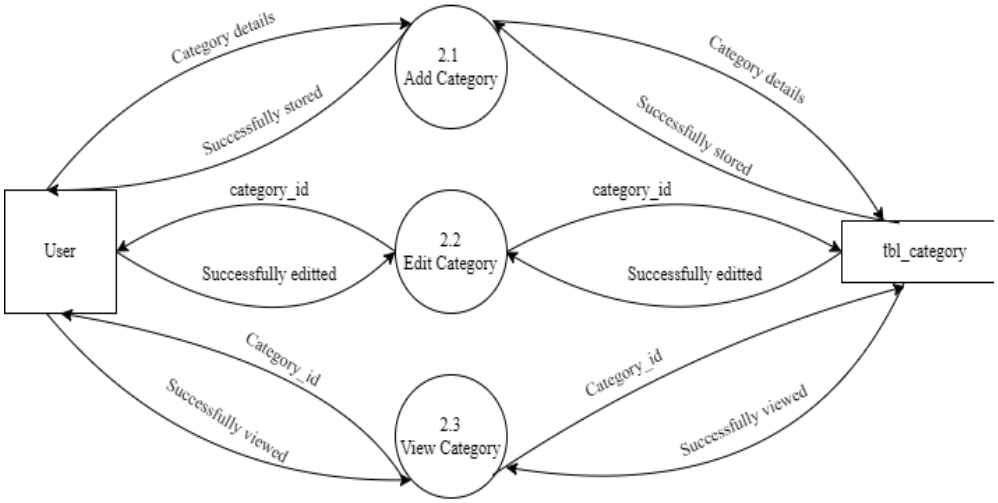
Level 1 DFD Showing Asset Management System



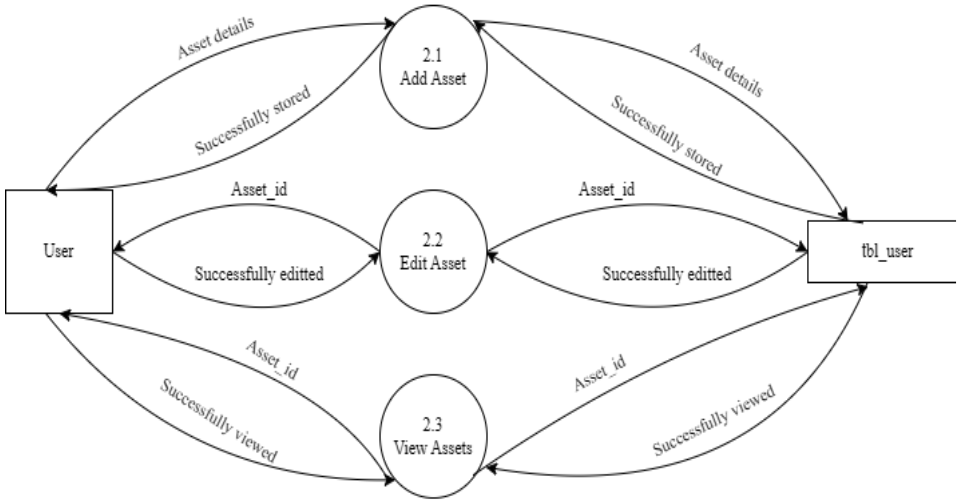
Level 2 DFD Showing User Management System



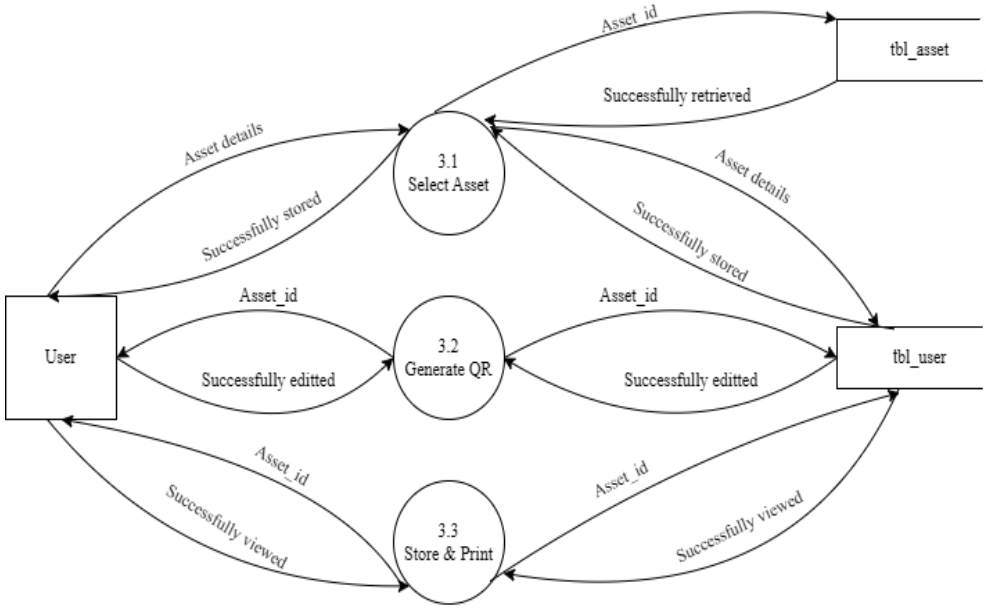
Level 2 DFD Showing User Management System



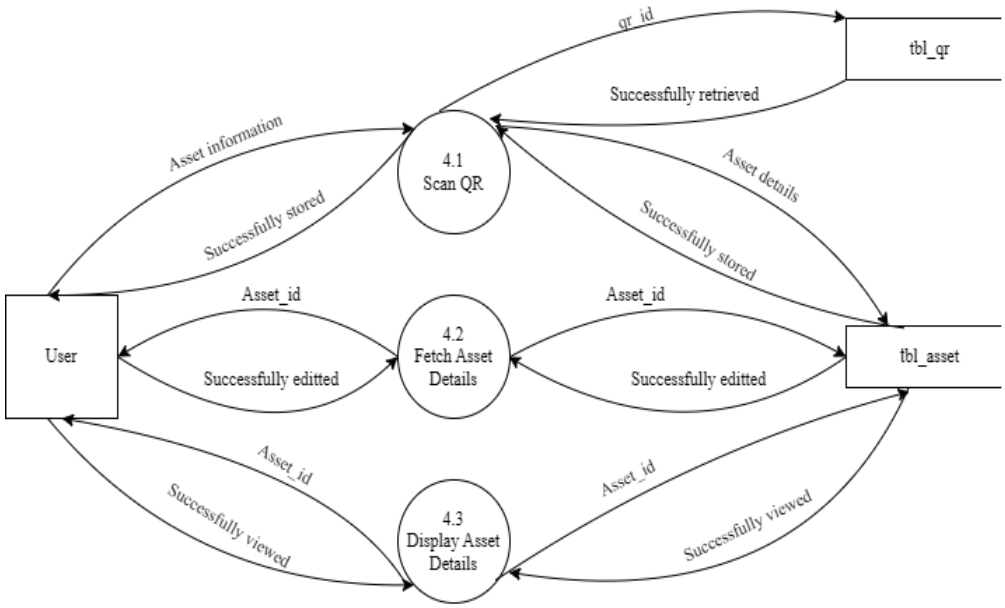
Level 2 DFD Showing Asset Management



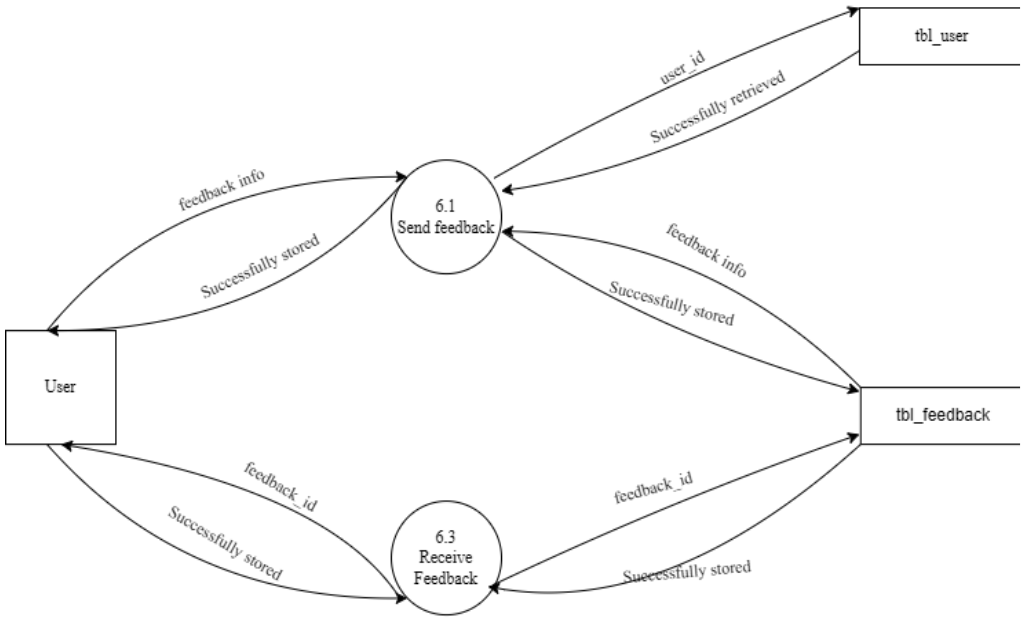
Level 2 DFD Showing QR Generation



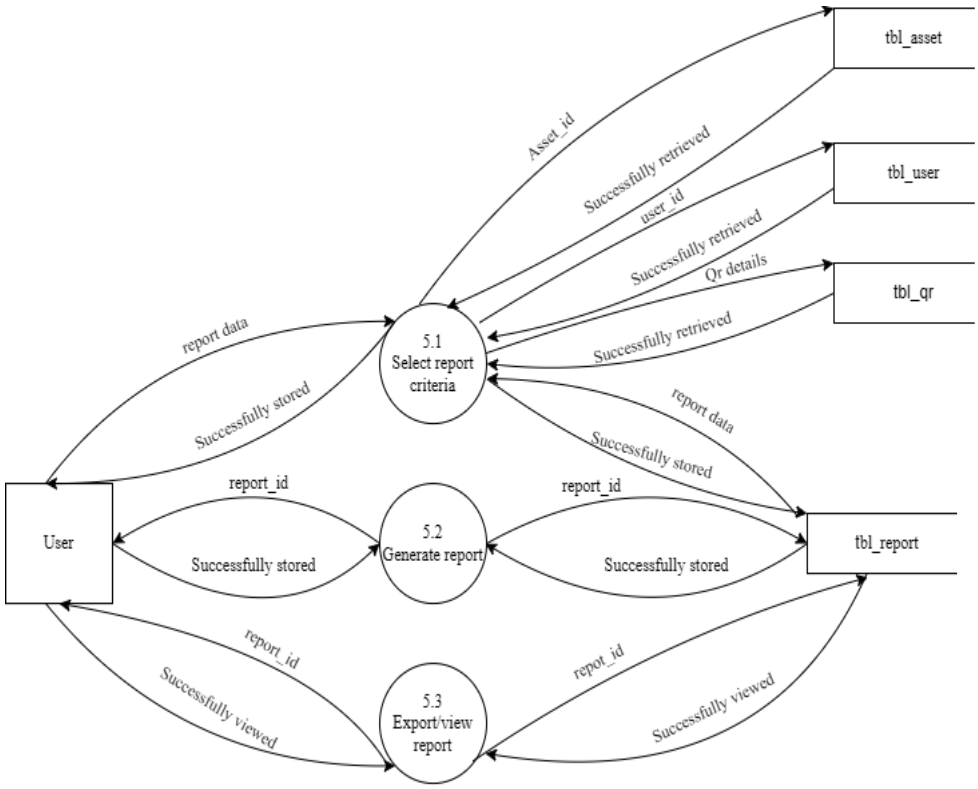
Level 2 DFD Showing Scan and view



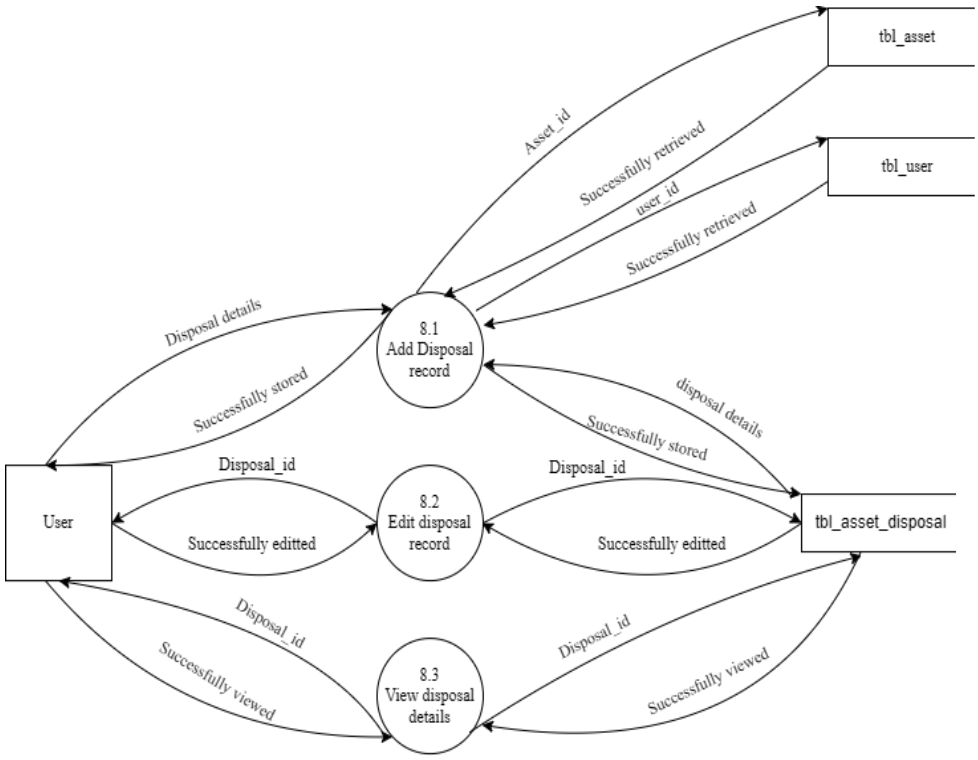
Level 2 DFD Showing Feedback



Level 2 DFD Showing Report Generation



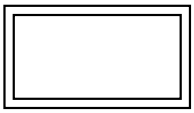
Level 2 DFD Showing Asset expiry



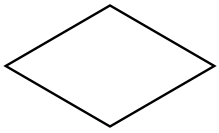
3.3 ENTITY RELATIONSHIP DIAGRAM

The ER model provides a conceptual way to perceive the real world, representing it as a collection of entities and the relationships between them. Central to this model is the Entity-Relationship diagram, a visual tool used to illustrate data components. ER modelling is extensively applied in the initial design phase of database applications, and its concepts are fundamental to various database design tools.

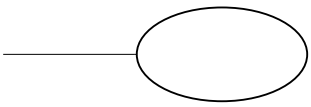
Entity Type



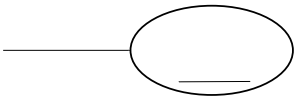
Weak Entity Type



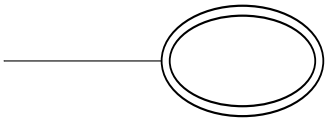
Relationship Type



Attribute

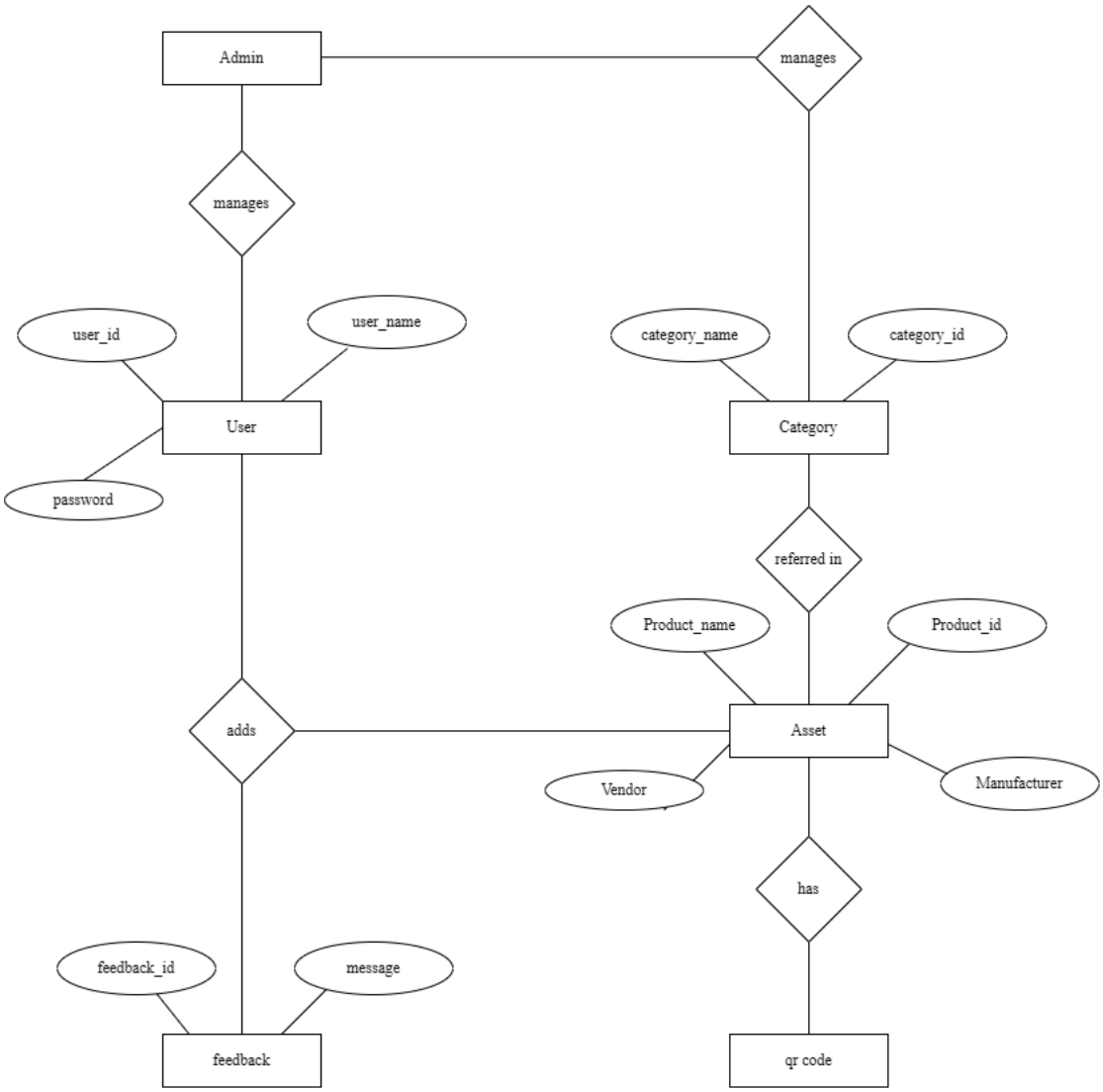


Key attribute



Multivalued Attribute

Entity Relationship diagram for Asset Management System



4.1 INPUT DESIGN

Input design is the process of converting user-oriented data into a form that can be processed accurately by the Asset Management System. Proper input design is essential to ensure data correctness, reduce errors, and improve overall system efficiency. In this system, inputs are collected from administrators and users through structured web forms.

The quality of input directly affects the quality of output generated by the system. Therefore, special attention has been given to designing input forms that are simple, clear, and user-friendly. The system captures essential data related to user accounts, asset details, categories, QR code information, feedback, and asset disposal records.

Input design in the Asset Management System ensures that all required data is collected in a consistent and organized manner. Mandatory fields are clearly indicated, and validations are applied to prevent incorrect or incomplete data entry.

The following factors were considered during input design:

Field Length Specification

Each input field has a predefined length to maintain uniformity and prevent invalid data entry. For example, email fields, identification numbers, and text fields have appropriate length constraints.

Field Sequence

Input fields are arranged in a logical order that follows the natural flow of data entry. This helps users enter information easily without confusion.

Data Format Validation

Input fields enforce specific formats such as date formats for purchase and expiry dates, numeric validation for price and warranty period, and proper format for email addresses.

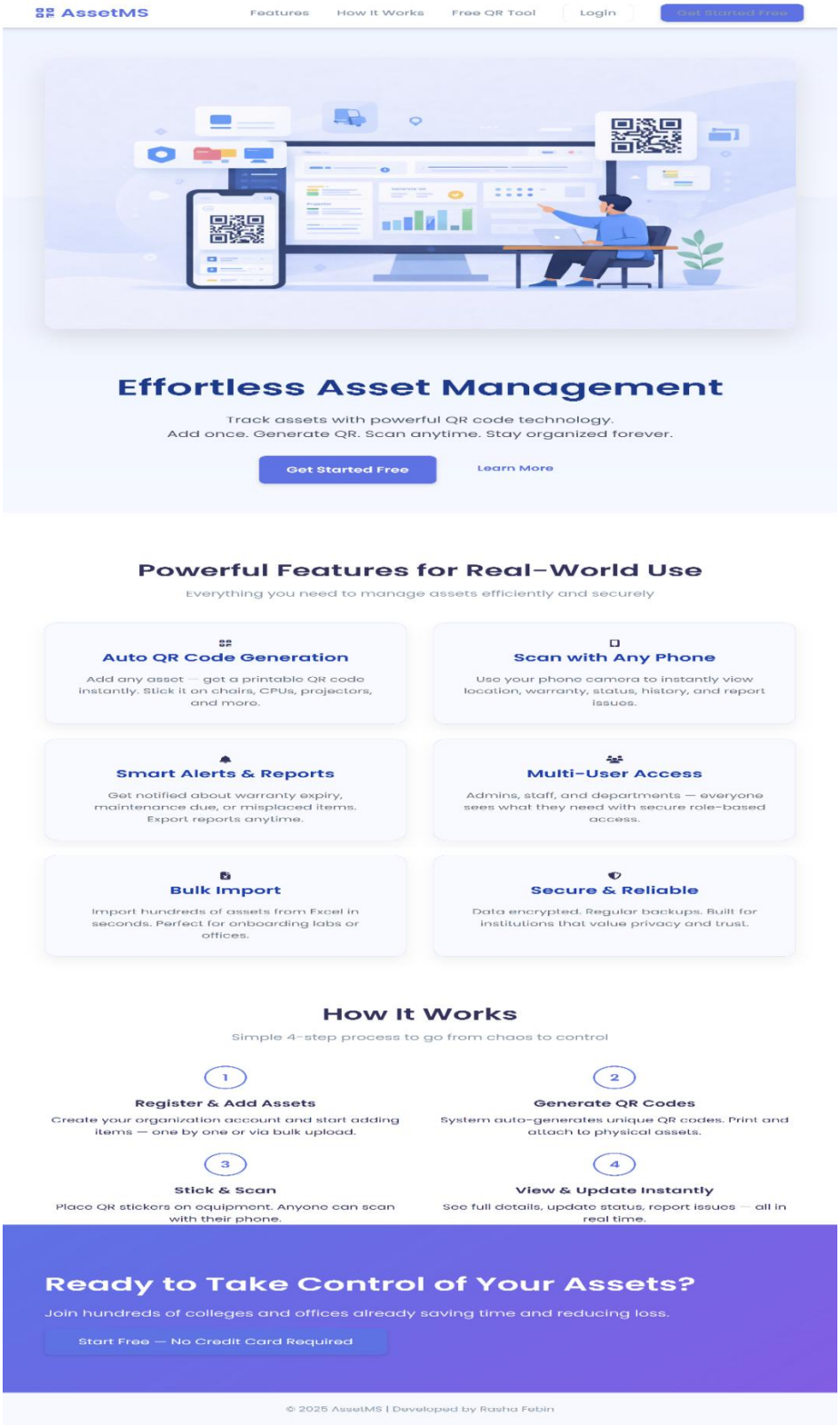
Most of the data in the system is entered through well-structured web forms. Dropdown menus are used for predefined values such as asset categories and user roles, while text fields are used for asset names, locations, and descriptions. This approach reduces manual typing errors and improves data accuracy.

To minimize errors, validation is implemented at both client-side and server-side levels. For example, while adding a new asset, the system ensures that mandatory fields such as asset name, category, purchase date, and expiry date are filled. Similarly, during user registration, the system validates email format and password requirements.

The Asset Management System primarily uses menus, formatted forms, and prompts for data entry. Menus guide users to appropriate sections, forms provide structured input areas, and prompts display validation or error messages whenever incorrect data is entered. This ensures reliable and efficient input handling throughout the system.

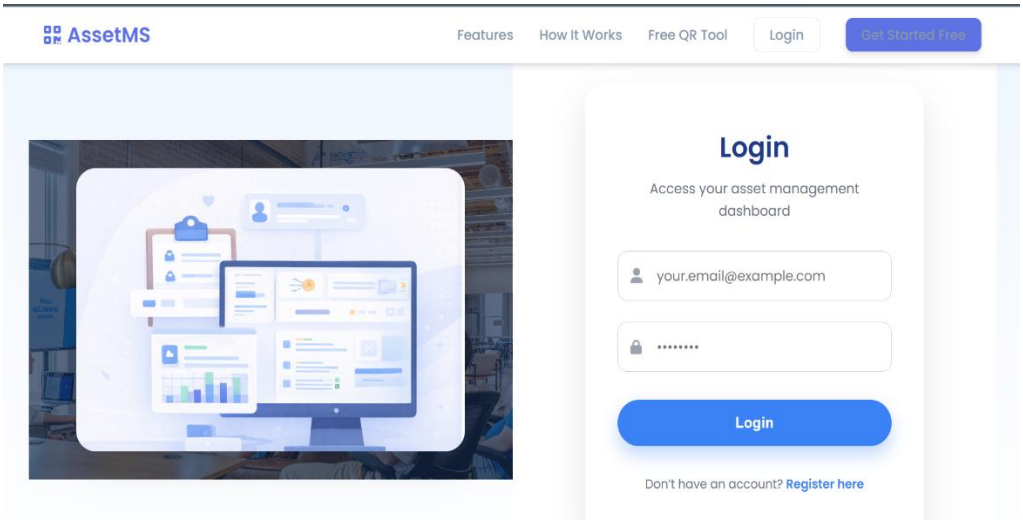
Home Page

Description: This is the homepage for all users.



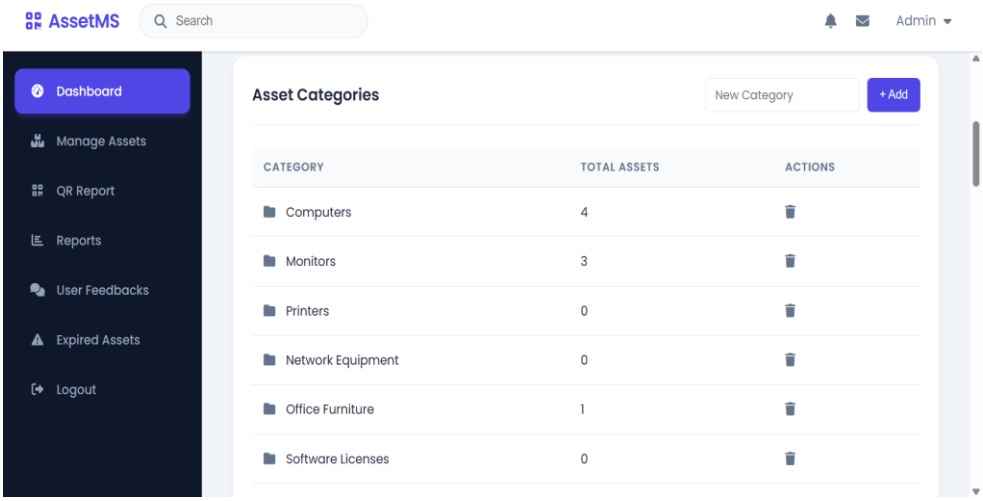
Login Form

Description: This is the login page for all users



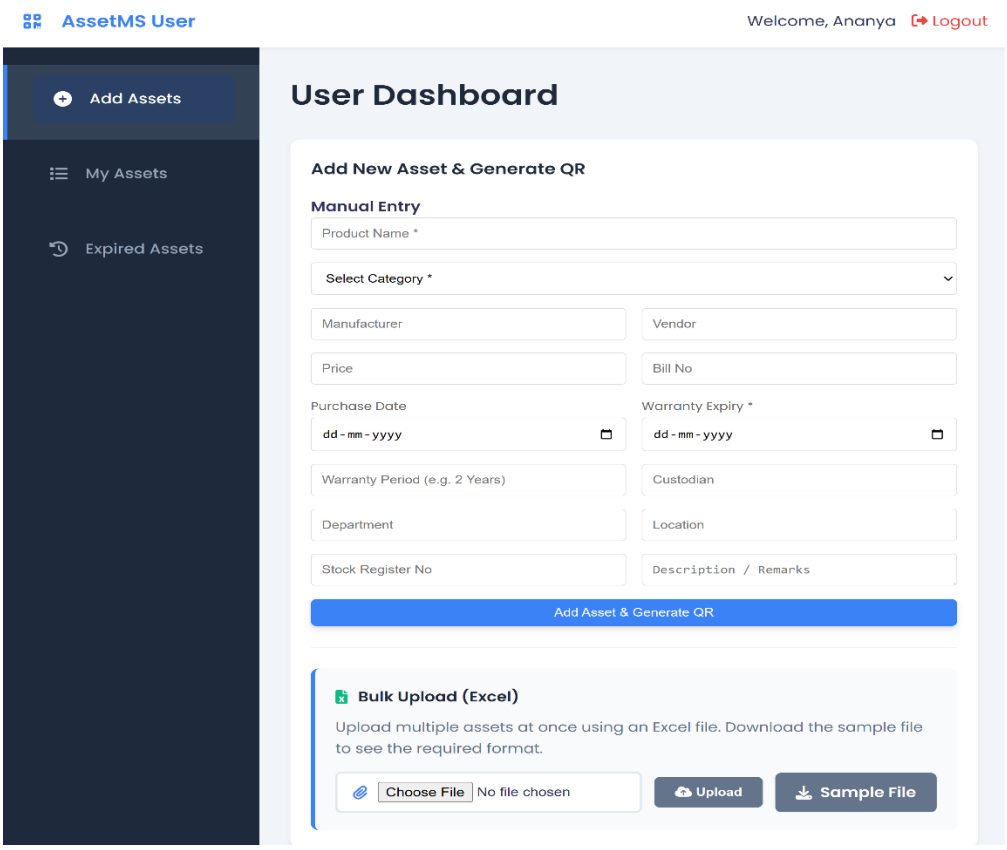
Category Management

Description: Add category



Asset Management

Description: Add asset details



4.2 OUTPUT DESIGN

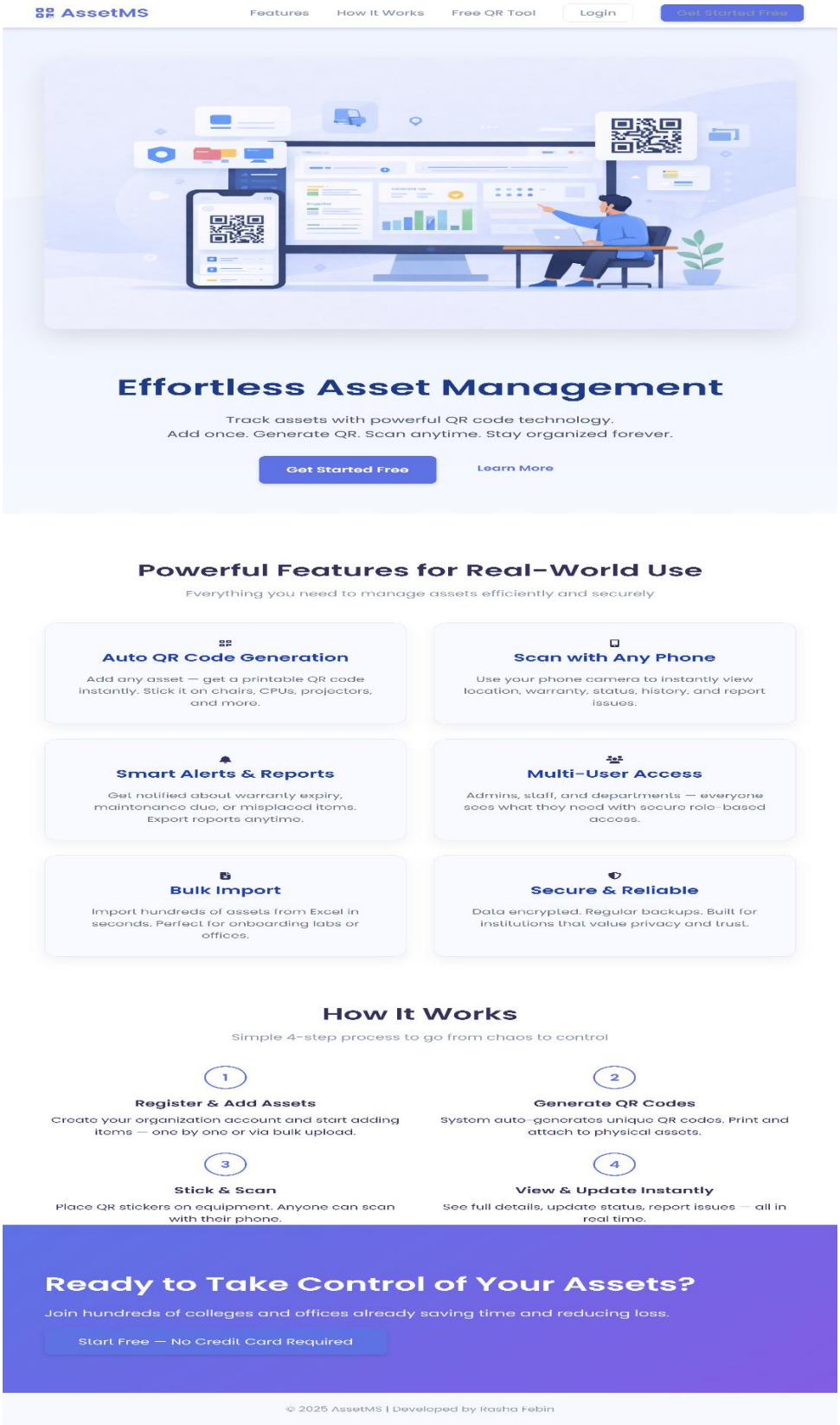
Output design is one of the most important aspects of an information system, as it determines how effectively the processed data is presented to the users. In the Asset Management System, outputs are designed to be clear, accurate, and easy to understand so that administrators and users can quickly interpret the information displayed.

The usefulness of the system largely depends on the quality of outputs it produces. Therefore, output design has been given significant importance in this project. The system provides outputs in the form of dashboards, tables, QR codes, reports, and confirmation messages, ensuring that information is presented in a structured and meaningful way.

During the analysis phase, various output requirements were identified, such as asset details, category-wise asset lists, QR code displays, feedback responses, expiry alerts, and reports. These outputs are mainly displayed on the screen for real-time usage, while some outputs such as reports can be exported for future reference.

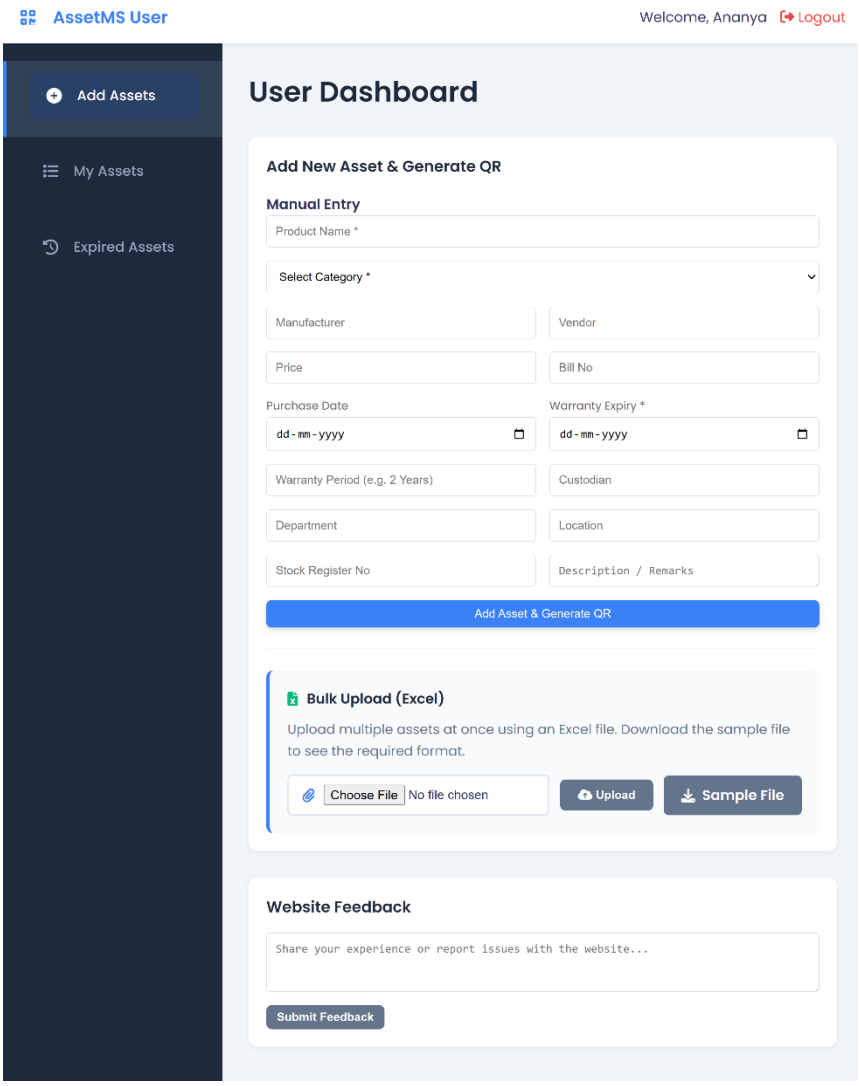
Home Page

Description: This is the homepage for all users



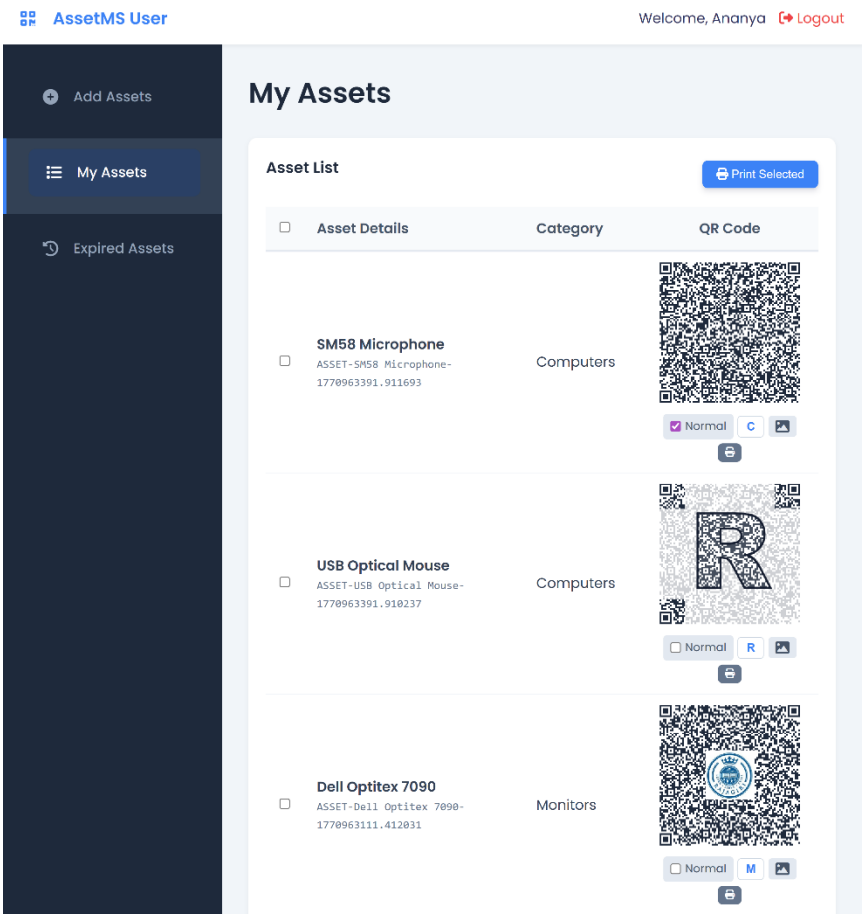
User dashboard

Description: Page showing user dashboard



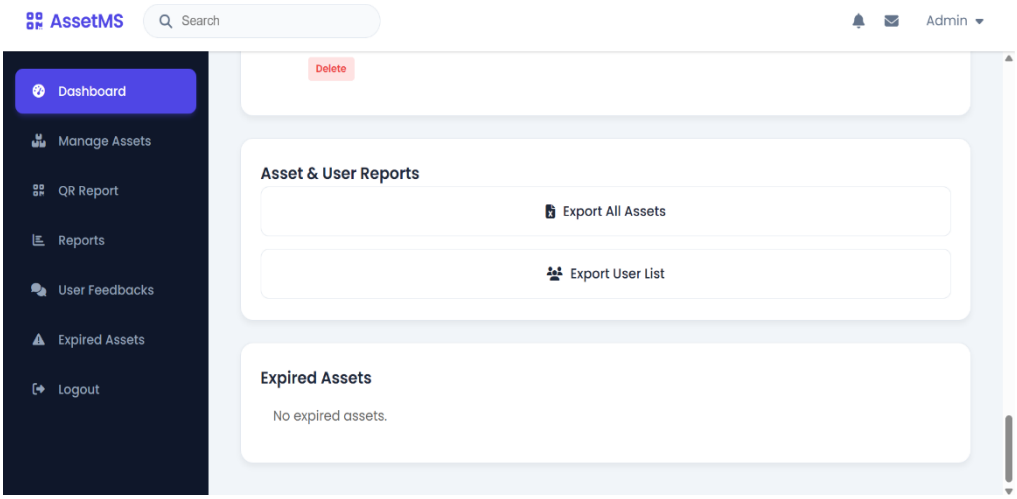
QR Generation

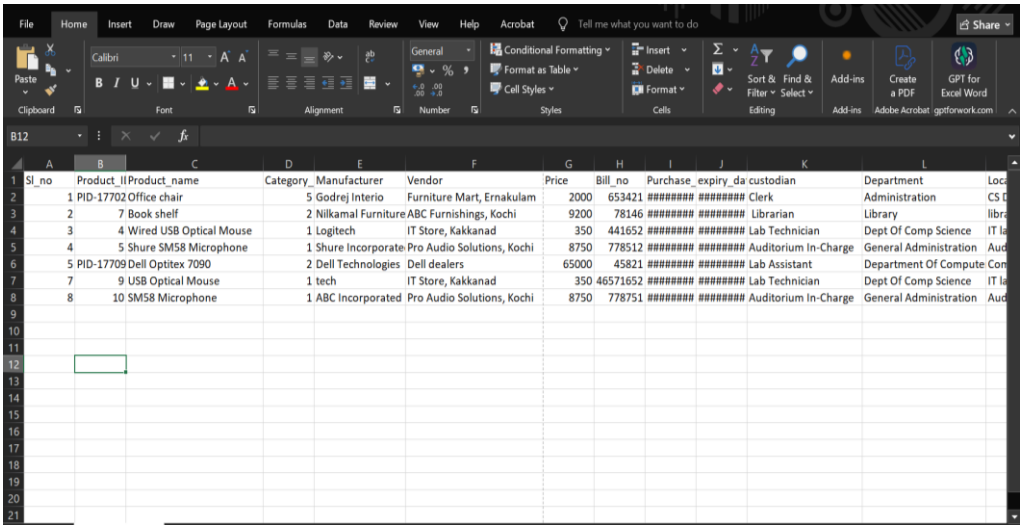
Description: Page showing all the qr codes generated by users as normal qr code , as an alphabet or an image



Report generation

Description: Report generation in the admin page for both assets and list of users

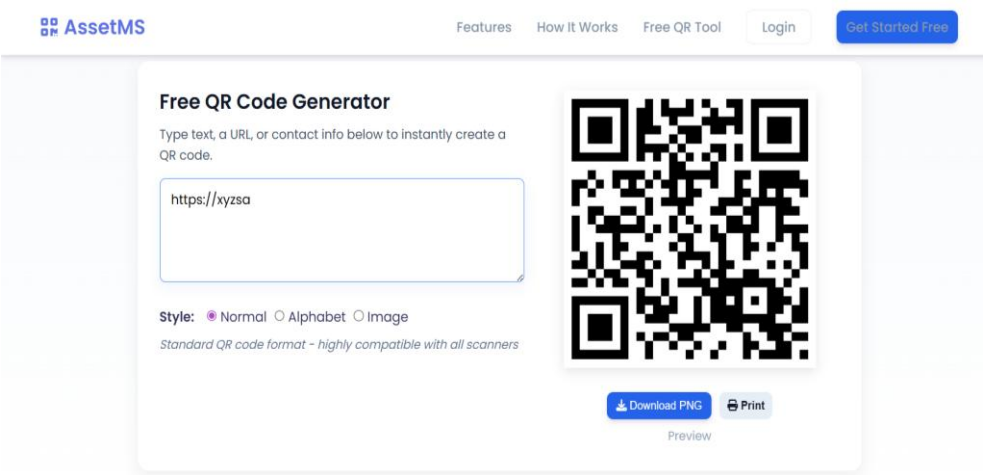




Sl_no	Product_II	Product_name	Category_	Manufacturer	Vendor	Price	Bill_no	Purchase_	expiry_da	custodian	Department	Loc
1	PID-17702	Office chair	5	Godrej Interio	Furniture Mart, Ernakulam	2000	653421	########	########	Clerk	Administration	CS D
2	7	Book shelf	2	Nilkamal Furniture	ABC Furnishings, Kochi	9200	78146	########	########	Librarian	Library	libra
3	4	Wired USB Optical Mouse	1	Logitech	IT Store, Kakkanad	350	441652	########	########	Lab Technician	Dept Of Comp Science	IT la
4	5	Shure SM58 Microphone	1	Shure Incorporate	Pro Audio Solutions, Kochi	8750	778512	########	########	Auditorium In-Charge	General Administration	Aud
5	PID-17709	Dell Optitex 7090	2	Dell Technologies	Dell dealers	65000	45821	########	########	Lab Assistant	Department Of Compute	Com
7	9	USB Optical Mouse	1	tech	IT Store, Kakkanad	350	46571652	########	########	Lab Technician	Dept Of Comp Science	IT la
8	10	SM58 Microphone	1	ABC Incorporated	Pro Audio Solutions, Kochi	8750	778751	########	########	Auditorium In-Charge	General Administration	Aud

Free qr code tool

Description: Free qr code tool for the user that converts any information they enter to qr codes



4.3 DATABASE DESIGN

4.3.1 Normalization

Normalization is the process of organizing data in a database to minimize redundancy and dependency. Proper normalization helps avoid data anomalies such as duplication, inconsistency, and loss of information. In the Asset Management System, normalization is applied to ensure that data is stored efficiently and maintained easily.

A poorly designed database may lead to the following problems:

- Repetition of data
- Inconsistency in data values
- Difficulty in updating records
- Loss of important information

To overcome these issues, the database tables in this system are designed following normalization principles.

First Normal Form (1NF)

A table is said to be in First Normal Form if all its attributes contain atomic (indivisible) values and there are no repeating groups. In this system, all tables are designed so that each field contains a single value only.

Second Normal Form (2NF)

A table is in Second Normal Form if it is in First Normal Form and all non-key attributes are fully dependent on the primary key. In the Asset Management System, each table has a clearly defined primary key, and all attributes depend entirely on it.

Third Normal Form (3NF)

A table is in Third Normal Form if it is in Second Normal Form and there are no transitive dependencies. This means non-key attributes do not depend on other non-key attributes. The database tables in this system follow Third Normal Form to ensure data consistency and reliability.

TABLE DESIGN

Table 1: tbl_user

Description: Table showing user details

Field	Data Type	Constraint	Description
login_id	int(11)	PRIMARY KEY, AUTO_INCREM E NT, NOT NULL	Login record id
user_name	varchar(5 0)	NOT NULL	Display name of the user
user_type	varchar(2 0)	NOT NULL	Type of user (admin/faculty/student/coordina t or/hod)
user_id	varchar(2 0)	UNIQUE, NOT NULL	Unique user identifier
password_ h ash	varchar(2 5 5)	NOT NULL	Hashed password (bcrypt)
status	tinyint(1)	NOT NULL, DEFAULT 1	Account status (1=active, 0=inactive)

Table 2: tbl_category

Description: Table Showing category details

Field	Data Type	Constraint	Description
Category_id	int	Primary key	Category id
Category_name	Varchar	NOT NULL	Name of the category
Category_description	varchar	NOT NULL	Description

Table 3: tbl_asset

Description : Table Showing asset details

Field	Data Type	Constraint	Description
Sl.no	integer	PRIMARY KEY, NOT NULL	Serial number
Asset_ID	date	NOT NULL	Product ID
Asset_name	date	NOT NULL	Product name
Category_ID	varchar(20)	NOT NULL	Category ID
Manufacturer	tinyint(1)	NOT NULL, DEFAULT 0	Name of the manufacturer of the product
Vendor	varchar	Not Null	Name of the vendor Of the product
Price	decimal	Not Null	Price of the product
Bill_no	integer	UNIQUE	Bill number of the product
Purchase_date	DATE	NOT NULL	Date of purchase of the product
expiry_date	DATE	NOT NULL	Date of expiry of the product
custodian	varchar	NOT NULL	Custodian/person in charge of the product
Department	varchar	NOT NULL	Department to which the product belongs
Location	varchar	NOT NULL	Location of the product
Warranty_period	DATE	NOT NULL	Warranty period of the product
Stock_register_no	integer	NOT NULL	Number of the product in stock register
Description	varchar	NOT NULL	Description of the product
Status	varchar	NOT NULL	Status of the product(in use/expired/scrapped)

Created_by	integer	Foreign key	User ID of the user who created the qr

Table 4: tbl_qr

Description : Table Showing qr details

Field	**Data Type**	**Constraint**	**Description**
Qr_id	integer	PRIMARY KEY, NOT NULL	Unique ID for each QR record in this table
Product_id	integer	F	Product ID
Qr_value	integer	NOT NULL, UNIQUE	The actual data encoded in the QR (for example, a unique token, asset code, or URL
Generated_at	DATE	NOT NULL	Date and time when this QR was generated

Table 5: tbl_feedback

Description : Table Showing feedback details

Field	**Data Type**	**Constraint**	**Description**
feedback_id	integer	PRIMARY KEY, NOT NULL	Feedback id
Asset_id	integer	FOREIGN KEY, NOT NULL	Asset id
message	TEXT	NOT NULL	Feedback message

Table 6: tbl_report

Description : Table Showing report details

Field	Data Type	Constraint	Description
report_id	varchar(10)	PRIMARY KEY, NOT NULL	Report id
asset_id	integer	FOREIGN KEY	Asset id
User_id	varchar(50)	FOREIGN KEY	Feedback mesage
Report_type	Varchar(50)	NOT NULL	Type of report("Asset summary","Expired assets","Assets by location")

Table 7: tbl_asset_expiry

Description : Table Showing Asset expiry/disposal

Field	Data Type	Constraint	Description
disposal_id	varchar(10)	PRIMARY KEY, NOT NULL	Disposal id
Asset_id	varchar(10)	FOREIGN KEY, NOT NULL	Asset id
Disposal_type	varchar(10)	FOREIGN KEY, NOT NULL	Disposal type
Stored_at	varchar(20)	NOT NULL, UNIQUE	Stored at
Sale_price	varchar(100)	NOT NULL	Sale price
Recipient_details	varchar(20)	NOT NULL	Details of the recipient
Approved_by	varchar	NOT NULL	If sold then approved by whom
Custodian_at_disposal	varchar	NOT NULL	Custodian at disposal
remarks	varchar	NOT NULL	remarks
Document_url	varchar	NOT NULL	Document url

5.1 INTRODUCTION

Testing is the process of evaluating a software system to ensure that it functions correctly and meets the specified requirements. The main objective of testing is to identify errors, verify system reliability, and ensure that the application performs efficiently under different conditions. In the Asset Management System, testing is carried out to confirm that all modules work properly and integrate smoothly with one another.

Testing plays a crucial role after system development and before deployment. It ensures that the system logic is correct, data is processed accurately, and outputs are generated as expected. The Asset Management System is tested at various levels to verify functionality, data flow, and user interaction.

Levels of Testing

5.1.1 Unit Testing

5.1.2 Integration Testing

5.1.3 Output Testing

5.1.4 Validation Testing

5.1.1 Unit Testing

Unit testing involves testing individual modules of the Asset Management System independently to ensure that each module functions correctly. Each unit is tested with valid and invalid inputs to verify proper handling of data and error conditions.

Modules such as User Management, Category Management, Asset Management, QR Generation, Feedback, and Report Generation are tested separately. For example, adding a new asset, generating a QR code, or

submitting feedback is tested individually to ensure correct data storage and confirmation messages.

5.1.2 Integration Testing

Integration testing is performed to ensure that different modules of the system work together as expected. This testing verifies data flow and interaction between interconnected modules.

For example, when an asset is added, it should be correctly linked with its category and QR code. Similarly, feedback submitted by users should be visible to administrators, and report generation should retrieve accurate data from multiple tables. Integration testing confirms that combined modules function smoothly without errors.

5.1.3 Output Testing

Output testing ensures that the system generates accurate and properly formatted outputs. This includes verifying dashboards, asset listings, QR code displays, reports, and confirmation messages.

For instance, when an asset is added, the system must display correct asset details. Generated QR codes should correspond to the correct asset. Reports should display accurate and up-to-date information. This testing ensures clarity and correctness of system outputs.

5.1.4 Validation Testing

Validation testing checks whether the system meets the functional requirements and user expectations. It ensures that the complete workflow of the system operates correctly from start to finish.

For example, user registration followed by login, asset addition, QR code generation, scanning and viewing asset details, and feedback submission are tested as complete processes. This confirms that the system behaves as intended and satisfies user requirements.

Unit Testing

Module	Procedure	Expected Result	Actual Result	Status
User Management	Enter valid user details and submit	User should be registered successfully	Registration confirmation message displayed	Pass
Category Management	Enter category name and save	Category should be stored in database	Category added successfully	Pass
Asset Management	Enter asset details and submit	Asset details should be saved	Asset added confirmation displayed	Pass
QR Generation	Generate QR code for asset	QR code should be generated	QR code generated correctly	Pass
Scan and View Asset	Scan QR code	Asset details should be displayed	Asset details displayed correctly	Pass
Feedback	Submit feedback message	Feedback should be recorded	Feedback confirmation shown	Pass
Report Generation	Generate asset report	Report should be generated	Report generated successfully	Pass

Integration Testing

Module	Procedure	Expected Result	Actual Result	Status
Login and Dashboard	User should login and be redirected to correct dashboard	User should be redirected to correct dashboard	Correct dashboard displayed	Pass
Asset and Category	Link asset to selected category	Asset should be linked to selected category	Asset linked successfully	Pass
Asset and QR Code	Generate QR code for asset	QR code should map to correct asset	QR-asset mapping successful	Pass
Feedback and Admin Panel	Submit feedback as user and view as admin	Feedback should be visible to admin	Feedback displayed to admin	Pass
Report Generation	Generate reports	Reports should display accurate data	Reports generated correctly	Pass

Validation Testing

Form	Expected Result	Actual Result	Status
Login Form	Accept valid credentials and reject invalid ones	Valid login allowed, invalid rejected	Pass
User Registration Form	Allow only valid inputs	Errors shown for invalid data	Pass
Asset Entry Form	Mandatory fields must be filled	Validation applied correctly	Pass
QR Scan	Display correct asset details	Correct asset details shown	Pass
Feedback Form	Accept non-empty feedback	Feedback submitted successfully	Pass
Report Generation	Generate correct reports	Accurate reports displayed	Pass

6.1 INTRODUCTION

Implementation is the stage in which the theoretical design of the Asset Management System is transformed into a fully functional working system. This phase plays a crucial role in ensuring that the system operates according to the defined requirements and objectives. During implementation, all modules of the system are developed, integrated, and tested to ensure smooth operation.

The implementation phase involves setting up the development environment, configuring the database, integrating frontend and backend components, and ensuring proper communication between different modules. Since the system includes features such as user management, asset management, QR code generation, scanning, reporting, and feedback handling, careful coordination of all components is required.

Before deploying the system for actual use, extensive testing is performed to identify and fix errors. Only after successful testing and validation is the system considered ready for real-world usage. This phase ensures that the Asset Management System is reliable, secure, and user-friendly.

6.2 INSTALLATION PROCEDURE

The installation procedure describes the steps required to install and run the Asset Management System on a local or server environment. The system can be deployed either on a local machine for testing purposes or on a server for actual use.

The installation process begins with setting up the required software environment. Python must be installed along with the Flask framework to run the backend of the application. MySQL is installed and configured to store system data such as user details, asset records, categories, feedback, and reports.

Once the software environment is ready, the project source files are placed in the working directory. The database is created using the provided SQL scripts, and required tables are initialized. Initial data such as admin users and categories can be inserted to prepare the system for use.

After database setup, the Flask application is executed, and the system can be accessed through a web browser. The system is then tested to ensure that all modules function correctly and that data is properly stored and retrieved from the database.

6.3 IMPLEMENTATION PLAN

The implementation plan outlines the step-by-step approach followed to deploy the Asset Management System successfully. A systematic plan ensures smooth transition from development to usage without major disruptions.

The implementation plan includes the following steps:

System Setup

Installation of required software tools such as Python, Flask, MySQL, and code editor. Configuration of the development environment.

Database Configuration

Creation of the database and tables, establishing relationships between tables, and inserting initial records such as admin details and asset categories.

Module Integration

Integration of all modules including User Management, Asset Management, QR Generation, Scan and View Assets, Feedback, Reports, and Asset Expiry/Disposal.

Testing Phase

Conducting unit testing, integration testing, output testing, and validation testing to ensure system reliability.

Deployment

Making the system available for users after successful testing.

Maintenance and Support

Monitoring system performance and making necessary updates or improvements when required.

This planned approach ensures that the Asset Management System is implemented efficiently and operates smoothly in a real-world environment.

7.1 FUTURE ENHANCEMENT

Although the Asset Management System fulfils the current requirements effectively, there is considerable scope for future improvements. These enhancements can further improve system performance, usability, and scalability.

Some possible future enhancements include:

1. **Mobile Application Support**
   Developing a mobile application to allow users to scan QR codes and view asset details more conveniently.
2. **Advanced QR Scanning**
   Integrating high-speed QR scanning and offline scanning features for areas with limited internet access.
3. **Automated Notifications**
   Sending email or SMS alerts for asset expiry, warranty completion, and disposal approvals.
4. **Role Expansion**
   Adding more user roles such as auditors or department heads with customized access permissions.
5. **Advanced Reporting and Analytics**
   Generating graphical reports and analytics for better decision-making and asset planning.
6. **Cloud Deployment**
   Hosting the system on a cloud platform to improve accessibility, scalability, and data backup.
7. **Integration with Other Systems**
   Integrating the asset management system with accounting or inventory systems for better coordination.

These future enhancements can make the system more robust, flexible, and suitable for large-scale organizational use.

BIBLIOGRAPHY

The following references were used during the development of the Asset Management System:

- **Flask Documentation**
  Flask: The Python Microframework. Retrieved from
  https://flask.palletsprojects.com/
- **SQLAlchemy Documentation**
  SQLAlchemy: The Database Toolkit for Python. Retrieved from
  https://www.sqlalchemy.org/
- **Bootstrap Documentation**
  Bootstrap: Frontend Framework for Responsive Web Design. Retrieved from
  https://getbootstrap.com/
- **Font Awesome**
  Font Awesome: Icons for Web Projects. Retrieved from
  https://fontawesome.com/
- **Python Documentation**
  Python: Official Documentation. Retrieved from
  https://docs.python.org/
- **Stack Overflow**
  Community-driven Q&A for Programming. Retrieved from
  https://stackoverflow.com/
- **W3Schools**
  Tutorials for HTML, CSS, and JavaScript. Retrieved from
  https://www.w3schools.com/
- **MDN Web Docs**
  Mozilla Developer Network: Web Development Resources. Retrieved from
  https://developer.mozilla.org/
- **SQLite Documentation**
  SQLite: Lightweight Database Engine. Retrieved from
  https://www.sqlite.org/

- **QR Code Generation Libraries**

  QR Code Libraries for Python. Retrieved from

  https://pypi.org/project/qrcode/

APPENDICES

APPENDIX A

Sample Source Code/Pseudo Code

App.py Code

```
from flask import Flask, render_template, request, redirect, session, url_for, flash, send_file
from flask_bcrypt import Bcrypt
from flask_cors import CORS
from db import get_db_connection
import pandas as pd
import qrcode
import io
import base64
from datetime import datetime
app = Flask(__name__)
app.secret_key = "secret123"

bcrypt = Bcrypt(app)
CORS(app)

def get_db():
    """Get a database connection for the current request."""
    if 'db' not in g:
        try:
            g.db = get_db_connection()
```

```
        except Exception as e:
            print(f"Database connection failed: {e}")
            flash("Database connection error. Please try again later.", "error")
            raise
    return g.db

@app.teardown_appcontext
def close_db(e=None):
    """Close the database connection at the end of the request."""
    db = g.pop('db', None)
    if db is not None:
        try:
            db.close()
        except Exception as close_error:
            print(f"Error closing database: {close_error}")
@app.route("/create-test-user")
def create_test_user():
    db = get_db()
    cur = db.cursor()

    hashed = bcrypt.generate_password_hash("admin123").decode('utf-8')

    cur.execute("""
        INSERT INTO tbl_user (User_name, Email, password_hash, user_type)
        VALUES (%s, %s, %s, %s)
    """, ("admin", "admin@gmail.com", hashed, "Admin"))  # Changed to use
bcrypt hash
```

```
    db.commit()

    return "Test user created"
@app.route("/")
def home():
    return render_template("home.html")

@app.route("/free-qr")
def free_qr():
    return render_template("free_qr.html")

@app.route("/set-admin-password")
def set_admin_password():
    db = get_db()
    cur = db.cursor()

    hashed = bcrypt.generate_password_hash("admin123").decode('utf-8')

    cur.execute("""
        UPDATE tbl_user
        SET password_hash = %s
        WHERE Email = %s
    """, (hashed, "admin@gmail.com"))  # Changed to use bcrypt hash

    db.commit()
```

```
    return "Admin password set"


# ---------------- REGISTER ----------------
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = request.form["full_name"]
        email = request.form["email"]
        password = request.form["password"]
        user_type = request.form["user_type"]

        hashed_pw = bcrypt.generate_password_hash(password).decode("utf-8")

        db = get_db()
        cursor = db.cursor()

        cursor.execute(
            "INSERT INTO tbl_user (User_name, Email, password_hash, user_type) VALUES (%s, %s, %s, %s)",
            (username, email, hashed_pw, user_type)
        )
        db.commit()

        return redirect("/login")

    return render_template("register.html")
```

```
# ---------------- LOGIN ----------------
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        email = request.form["email"]
        password = request.form["password"]

        db = get_db()
        cursor = db.cursor(dictionary=True, buffered=True)

        cursor.execute("SELECT * FROM tbl_user WHERE Email=%s", (email,))
        user = cursor.fetchone()

        if user and bcrypt.check_password_hash(user["password_hash"],
password):
            session["user_id"] = user["User_id"]
            session["username"] = user["User_name"]
            session["role"] = user["user_type"]
            return redirect("/dashboard")

        flash("Invalid email or password. Please try again.", "error")
        return redirect("/login")

    return render_template("login.html")

# ---------------- DASHBOARD ----------------
@app.route("/dashboard")
```

```
def dashboard():
    if "user_id" not in session:
        return redirect("/login")

    if session.get("role") == "Admin":
        return redirect("/admin/dashboard")

    # User Dashboard Logic
    db = get_db()
    cur = db.cursor(dictionary=True, buffered=True)

    # Fetch categories for the dropdown
    cur.execute("SELECT * FROM tbl_category")
    categories = cur.fetchall()

    # Fetch assets to display to the user
    cur.execute("SELECT a.*, c.Category_name FROM tbl_asset a LEFT JOIN
tbl_category c ON a.Category_ID = c.Category_id ORDER BY a.Sl_no DESC")
    assets = cur.fetchall()

    # Fetch expired assets
    cur.execute("SELECT * FROM tbl_asset WHERE expiry_date <
CURDATE()")
    expired_assets = cur.fetchall()

    return render_template(
        "user_dashboard.html",
        username=session["username"],
```

```
        categories=categories,
        assets=assets,
        expired_assets=expired_assets
    )

@app.route("/admin/dashboard")
def admin_dashboard():
    if "user_id" not in session or session["role"] != "Admin":
        return redirect("/login")

    db = get_db()
    cur = db.cursor(dictionary=True, buffered=True)

    # 1. KPIs
    cur.execute("SELECT COUNT(*) as count FROM tbl_asset")
    total_assets = cur.fetchone()['count'] if cur.rowcount > 0 else 0

    cur.execute("SELECT COUNT(*) as count FROM tbl_category")
    total_categories = cur.fetchone()['count'] if cur.rowcount > 0 else 0

    cur.execute("SELECT COUNT(*) as count FROM tbl_feedback")
    pending_requests = cur.fetchone()['count'] if cur.rowcount > 0 else 0

    cur.execute("SELECT COUNT(*) as count FROM tbl_asset WHERE
expiry_date < CURDATE()")
    expired_assets_count = cur.fetchone()['count'] if cur.rowcount > 0 else 0
```

```
        # 2. Categories with Asset Counts
        cur.execute("""
                SELECT c.Category_id, c.Category_name, COUNT(a.Sl_no) as asset_count
            FROM tbl_category c
            LEFT JOIN tbl_asset a ON c.Category_id = a.Category_ID
            GROUP BY c.Category_id
        """)
        categories = cur.fetchall()

        # 3. Recent Feedbacks
        cur.execute("""
            SELECT f.*, u.User_name
            FROM tbl_feedback f
            JOIN tbl_user u ON f.user_id = u.User_id
            ORDER BY f.feedback_id DESC LIMIT 5
        """)
        feedbacks = cur.fetchall()

        # 4. Expired Assets
        cur.execute("""
            SELECT * FROM tbl_asset
            WHERE expiry_date < CURDATE()
            LIMIT 5
        """)
        expired_assets = cur.fetchall()
```

```
    # 5. All Assets for QR Report
    cur.execute("""
        SELECT a.*, c.Category_name
        FROM tbl_asset a
        LEFT JOIN tbl_category c ON a.Category_ID = c.Category_id
        ORDER BY a.Sl_no DESC
    """)
    all_assets = cur.fetchall()

    return render_template(
        "dashboard.html",
        username=session["username"],
        total_assets=total_assets,
        total_categories=total_categories,
        pending_requests=pending_requests,
        expired_assets_count=expired_assets_count,
        categories=categories,
        feedbacks=feedbacks,
        expired_assets=expired_assets,
        all_assets=all_assets
    )

# ---------------- ADMIN ACTIONS ----------------

@app.route("/admin/add-category", methods=["POST"])
def add_category():
```

```
        if "user_id" not in session or session["role"] != "Admin": return redirect("/login")
        category_name = request.form.get("category_name")
        if category_name:
            db = get_db()
            cur = db.cursor()
            cur.execute("INSERT INTO tbl_category (Category_name) VALUES (%s)", (category_name,))
            db.commit()
            flash("Category added successfully!", "success")
        return redirect("/admin/dashboard")


    @app.route("/admin/delete-category/<int:id>")
    def delete_category(id):
        if "user_id" not in session or session["role"] != "Admin": return redirect("/login")
        db = get_db()
        cur = db.cursor()
        cur.execute("DELETE FROM tbl_category WHERE Category_id = %s", (id,))
        db.commit()
        flash("Category deleted.", "success")
        return redirect("/admin/dashboard")


    @app.route("/add-asset", methods=["POST"])
    def add_asset():
        if "user_id" not in session: return redirect("/login")
        # Manual Add
```

```
name = request.form.get("asset_name")
cat_id = request.form.get("category_id")
expiry = request.form.get("expiry_date")

# Additional Fields
manufacturer = request.form.get("manufacturer")
vendor = request.form.get("vendor")
price = request.form.get("price")
bill_no = request.form.get("bill_no")
purchase_date = request.form.get("purchase_date")
custodian = request.form.get("custodian")
department = request.form.get("department")
location = request.form.get("location")
warranty_period = request.form.get("warranty_period")
stock_register_no = request.form.get("stock_register_no")
description = request.form.get("description")

# Generate QR Data (Simple string for now)
qr_data = f"ASSET-{name}-{datetime.now().timestamp()}"
# Generate a Product ID if needed, or use random
prod_id = f"PID-{int(datetime.now().timestamp())}"

db = get_db()
cur = db.cursor()
cur.execute("""
    INSERT INTO tbl_asset (
        Product_ID, Product_name, Category_ID, Manufacturer, Vendor, Price,
```

```
                Bill_no, Purchase_date, expiry_date, custodian, Department, Location,
                Warranty_period, Stock_register_no, Description,
                Qr_code_value, Status, created_by
            ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
%s, %s, 'Active', %s)
        """, (
            prod_id, name, cat_id, manufacturer, vendor, price,
            bill_no, purchase_date, expiry, custodian, department, location,
            warranty_period, stock_register_no, description,
            qr_data, session['user_id']
        ))
        db.commit()
        flash("Asset added successfully!", "success")
        return redirect("/dashboard")

    @app.route("/upload-bulk", methods=["POST"])
    def upload_bulk():
        if "user_id" not in session: return redirect("/login")
        file = request.files.get('file')
        if file and file.filename.endswith(('.xls', '.xlsx')):
            try:
                df = pd.read_excel(file)
                # Replace NaN values with None (NULL in SQL)
                df = df.where(pd.notnull(df), None)

                db = get_db()
                cur = db.cursor()
```

```
                for _, row in df.iterrows():
                    # Generate QR Data
                                qr_data = f"ASSET-{row.get('Product_name')}-
{datetime.now().timestamp()}"


                    # Use provided Product_ID or generate one if missing
                    prod_id = row.get('Product_ID')
                    if not prod_id:
                        prod_id = f"PID-{int(datetime.now().timestamp())}"


                    cur.execute("""
                        INSERT INTO tbl_asset (
                         Product_ID, Product_name, Category_ID, Manufacturer, Vendor,
Price,
                            Bill_no, Purchase_date, expiry_date, custodian, Department,
Location,
                          Warranty_period, Stock_register_no, Description,
                          Qr_code_value, Status, created_by
                         ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s,
%s, %s, %s, %s, 'Active', %s)
                    """, (
                            prod_id, row.get('Product_name'), row.get('Category_ID'),
row.get('Manufacturer'),
                                row.get('Vendor'), row.get('Price'), row.get('Bill_no'),
row.get('Purchase_date'),
                        row.get('expiry_date'), row.get('custodian'), row.get('Department'),
row.get('Location'),
                              row.get('Warranty_period'), row.get('Stock_register_no'),
row.get('Description'),
                        qr_data, session['user_id']
```

```
                ))
                db.commit()
                flash("Bulk upload successful!", "success")
            except Exception as e:
                flash(f"Error uploading file: {str(e)}", "error")
        return redirect("/dashboard")


    @app.route("/submit-feedback", methods=["POST"])
    def submit_feedback():
        if "user_id" not in session: return redirect("/login")


        message = request.form.get("message")


        if message:
            db = get_db()
            cur = db.cursor()
             cur.execute("INSERT INTO tbl_feedback (user_id, message) VALUES
(%s, %s)",
                    (session["user_id"], message))
            db.commit()
            flash("Feedback submitted successfully.", "success")
        return redirect("/dashboard")


    @app.route("/admin/delete-feedback/<int:id>")
    def delete_feedback(id):
            if "user_id" not in session or session["role"] != "Admin": return
redirect("/login")
        db = get_db()
```

```
    cur = db.cursor()
    cur.execute("DELETE FROM tbl_feedback WHERE feedback_id=%s", (id,))
    db.commit()
    flash("Feedback deleted.", "success")
    return redirect("/admin/dashboard")


@app.route("/admin/export-report")
def export_report():
    if "user_id" not in session or session["role"] != "Admin": return redirect("/login")
    db = get_db()
    # Using pandas to fetch and create CSV
    df = pd.read_sql("SELECT * FROM tbl_asset", db)
    output = io.BytesIO()
    df.to_csv(output, index=False)
    output.seek(0)
    return send_file(output, mimetype="text/csv", as_attachment=True, download_name="asset_report.csv")


@app.route("/admin/export-user-report")
def export_user_report():
    if "user_id" not in session or session["role"] != "Admin": return redirect("/login")
    db = get_db()
    df = pd.read_sql("SELECT User_id, User_name, Email, user_type, Phone_no FROM tbl_user", db)
    output = io.BytesIO()
    df.to_csv(output, index=False)
```

```
    output.seek(0)
    return send_file(output, mimetype="text/csv", as_attachment=True, download_name="user_report.csv")


# ---------------- LOGOUT ----------------
@app.route("/logout")
def logout():
    session.clear()
    return redirect("/login")


if __name__ == "__main__":
    app.run(debug=True)
```

Login Page Code

```
{% extends "base.html" %}


{% block title %}Login | AssetMS{% endblock %}


{% block content %}
<section class="auth-section" style="background: linear-gradient(to bottom, #f0f7ff, #ffffff); min-height: 100vh;">
    <div class="auth-container" style="display: grid; grid-template-columns: 1fr 1fr; min-height: 100vh; align-items: center; max-width: 1400px; margin: 0 auto; padding: 2rem;">
        <!-- Left Illustration Side (Your Image) -->
        <div class="auth-image-side" style="display: flex; justify-content: center; align-items: center; padding: 2rem;">
            <img src="{{ url_for('static', filename='images/imglog.png') }}"
```

```
                    alt="Asset Management Dashboard Illustration"

                    style="max-width: 90%; border-radius: 1.5rem; box-shadow: 0
20px 40px rgba(0,0,0,0.1);">

            </div>


            <!-- Right Form Card -->

            <div class="auth-form-side" style="display: flex; justify-content:
center; align-items: center;">

                <div class="auth-card" style="background: white; padding: 3rem
2.5rem; border-radius: 1.5rem; box-shadow: 0 20px 40px rgba(0,0,0,0.1); width:
100%; max-width: 420px;">

                    <h2 style="font-size: 2rem; color: #1e3a8a; text-align: center;
margin-bottom: 0.5rem;">Login</h2>

                    <p style="text-align: center; color: #6b7280; margin-bottom:
2rem;">Access your asset management dashboard</p>


                    {% with messages =
get_flashed_messages(with_categories=true) %}

                        {% if messages %}

                            {% for category, message in messages %}

                                <div class="flash-message {{ category }}"
style="padding: 1rem; margin-bottom: 1rem; border-radius: 0.5rem; text-align:
center;">

                                    {{ message }}

                                </div>

                            {% endfor %}

                        {% endif %}

                    {% endwith %}


                    <form method="POST">
```

```
                    <div class="form-group icon-group" style="margin-bottom: 1.5rem; position: relative;">
                        <i class="fas fa-user" style="position: absolute; left: 1rem; top: 50%; transform: translateY(-50%); color: #9ca3af; font-size: 1.1rem;"></i>
                        <input type="email" name="email" placeholder="your.email@example.com" required
                            style="width: 100%; padding: 1rem 1rem 1rem 3rem; border: 1px solid #d1d5db; border-radius: 0.75rem; font-size: 1rem;">
                    </div>
                    <div class="form-group icon-group" style="margin-bottom: 2rem; position: relative;">
                        <i class="fas fa-lock" style="position: absolute; left: 1rem; top: 30%; color: #9ca3af;"></i>
                        <input type="password" name="password" placeholder="••••••••" required
                            style="width: 100%; padding: 1rem 1rem 1rem 3rem; border: 1px solid #d1d5db; border-radius: 0.75rem; font-size: 1rem;">
                    </div>
                    <button type="submit" class="btn-primary full-width"
                        style="background: #3b82f6; color: white; padding: 1rem; border-radius: 9999px; font-weight: 600; font-size: 1.1rem; border: none; cursor: pointer; box-shadow: 0 10px 20px rgba(59,130,246,0.3);">
                        Login
                    </button>
                </form>


                <p class="auth-footer" style="text-align: center; margin-top: 2rem; color: #6b7280;">
                    Don’t have an account? <a href="/register" style="color: #3b82f6; font-weight: 600;">Register here</a>
                </p>
            </div>
```

```
            </div>
        </div>
    </section>
    {% endblock %}
```

QR Code Generation Code

```
    document.addEventListener('DOMContentLoaded', function() {
        var qrContainers = document.querySelectorAll('.qr-code-target');
        qrContainers.forEach(function(container) {
            var input = container.parentElement.querySelector('.qr-letter-input');
            var letter = input ? input.value : 'A';
            generateQR(container, letter);
        });
    });

    function generateQR(container, letterOverride = null) {
        var qrText = container.getAttribute('data-qr');
        var customImgSrc = container.getAttribute('data-custom-img');
        var normalQRMode = container.getAttribute('data-normal-qr') ===
'true';

        // Determine letter to use if no image
        let letter = letterOverride;
        if (!letter && !customImgSrc && !normalQRMode) {
            const input = container.parentElement.querySelector('.qr-letter-
input');
            letter = input ? input.value.toUpperCase() : 'A';
        }
```

```
container.innerHTML = "";
if (!qrText) return;

// Create temporary container for the library
const tempDiv = document.createElement('div');
tempDiv.style.width = '400px';
tempDiv.style.height = '400px';
document.body.appendChild(tempDiv);

try {
    // Generate QR with High Error Correction
    new QRCode(tempDiv, {
        text: qrText,
        width: 400,
        height: 400,
        colorDark: "#1e293b",
        correctLevel: QRCode.CorrectLevel.H
    });

    // Wait for generation with retry logic
    let attempts = 0;
    const checkQRGeneration = () => {
        const qrCanvas = tempDiv.querySelector('canvas');
        const qrImg = tempDiv.querySelector('img');

        if ((qrCanvas || qrImg) && ((qrCanvas && qrCanvas.width > 0) ||
```

```
(qrImg && qrImg.width > 0 && qrImg.height > 0))) {

// Normal QR mode - standard QR without overlay
if (normalQRMode) {
    const canvas = document.createElement('canvas');
    canvas.width = 400;
    canvas.height = 400;
    const ctx = canvas.getContext('2d');
    ctx.fillStyle = '#FFFFFF';
    ctx.fillRect(0, 0, 400, 400);
    if (qrCanvas) ctx.drawImage(qrCanvas, 0, 0, 400, 400);
    else ctx.drawImage(qrImg, 0, 0, 400, 400);
    container.appendChild(canvas);
    document.body.removeChild(tempDiv);
    return;
}

// Letter Mode - QR with letter overlay
const canvas = document.createElement('canvas');
canvas.width = 400;
canvas.height = 400;
const ctx = canvas.getContext('2d');

// Draw Full QR Code (Base Layer)
ctx.fillStyle = '#FFFFFF';
ctx.fillRect(0, 0, 400, 400);
if (qrCanvas) ctx.drawImage(qrCanvas, 0, 0, 400, 400);
```

```
else ctx.drawImage(qrImg, 0, 0, 400, 400);

// Create "Spotlight" Mask Layer for LETTER mode
const maskCanvas = document.createElement('canvas');
maskCanvas.width = 400;
maskCanvas.height = 400;
const mCtx = maskCanvas.getContext('2d');

// Fill with semi-transparent white (The "Faded" area)
mCtx.fillStyle = 'rgba(255, 255, 255, 0.78)';
mCtx.fillRect(0, 0, 400, 400);

// Cut out the Letter shape
mCtx.globalCompositeOperation = 'destination-out';
const fontSize = 296;
mCtx.font = '900 ' + fontSize + 'px Arial, sans-serif';
mCtx.textAlign = 'center';
mCtx.textBaseline = 'middle';
mCtx.fillText(letter, 200, 216);
mCtx.globalCompositeOperation = 'source-over';

// Combine layers
ctx.clearRect(0, 0, 400, 400);
if (qrCanvas) ctx.drawImage(qrCanvas, 0, 0, 400, 400);
else ctx.drawImage(qrImg, 0, 0, 400, 400);
ctx.drawImage(maskCanvas, 0, 0);
```

```
                    // Draw letter stroke outline
                    ctx.font = '900 ' + fontSize + 'px Arial, sans-serif';
                    ctx.textAlign = 'center';
                    ctx.textBaseline = 'middle';
                    ctx.lineWidth = 6;
                    ctx.strokeStyle = 'rgba(15, 23, 42, 0.95)';
                    ctx.strokeText(letter, 200, 216);

                    // Restore Finder Patterns (Full Contrast) for scannability
                    const finderSize = 72;
                    const source = qrCanvas || qrImg;
                    ctx.drawImage(source, 0, 0, finderSize, finderSize, 0, 0,
finderSize, finderSize);
                    ctx.drawImage(source, 400-finderSize, 0, finderSize, finderSize,
400-finderSize, 0, finderSize, finderSize);
                    ctx.drawImage(source, 0, 400-finderSize, finderSize, finderSize,
0, 400-finderSize, finderSize, finderSize);

                    container.appendChild(canvas);
                    document.body.removeChild(tempDiv);

                } else if (attempts < 20) {
                    attempts++;
                    setTimeout(checkQRGeneration, 100);
                }
            };
            checkQRGeneration();
        } catch (error) {
```

```
        console.error('QR generation error:', error);
        container.innerHTML = "<span style='color:red;'>Error</span>";
        document.body.removeChild(tempDiv);
    }
}
```

APPENDIX B

Acronyms

ER – Entity Relationship

DFD – Data Flow Diagram

SQL – Structured Query Language

QR – Quick Response

1NF – First Normal Form

2NF – Second Normal Form

3NF – Third Normal Form